

Fin-GAN: Forecasting and Classifying Financial Time Series via Generative Adversarial Networks

Milena Vuletić^{*†◇}, Mihai Cucuringu^{†‡◇} and Felix Prenzell[†]

[†] Mathematical Institute, University of Oxford, Andrew Wiles Building, Woodstock Rd, Oxford OX2 6GG, UK

[‡] Department of Statistics, University of Oxford, 24-29 St Giles', Oxford OX1 3LB, UK

[◇] The Alan Turing Institute, 96 Euston Rd, London NW1 2DB, UK

[◇] Oxford-Man Institute of Quantitative Finance, University of Oxford, Eagle House, Walton Well Rd, Oxford OX2 6ED

We investigate the use of Generative Adversarial Networks (GANs) for probabilistic forecasting of financial time series. To this end, we introduce a novel economics-driven loss function for the generator. This newly designed loss function renders GANs more suitable for a classification task, and places them into a supervised learning setting, whilst producing full conditional probability distributions of price returns given previous historical values. Our approach moves beyond the point estimates traditionally employed in the forecasting literature, and allows for uncertainty estimates. Numerical experiments on equity data showcase the effectiveness of our proposed methodology, which achieves higher Sharpe Ratios compared to classical supervised learning models, such as LSTMs and ARIMA.

Keywords: GANs, financial returns, time series forecasting, classification.

JEL Classification: G17, C15, C22, C32, C45, C53

1. Introduction

Time series forecasting has been a core topic of interest for many years, spanning both industry and academia. Most real-world processes are naturally endowed with a time-series structure. In finance and economics, these can be the prices of financial instruments, inflation rates, and many other key macroeconomic indicators. Trading strategies are based on informed beliefs on whether the price of a certain financial instrument will exhibit an upwards or downwards move, along with the magnitude of the move. Hence, time series forecasting and classification are of utmost importance in financial settings. Traditionally, this task has been achieved using methods that rely solely on point estimates. In this work, our goal is to move beyond such classical approaches, and provide *probabilistic forecasts* for the future distribution of the target variable via Generative Adversarial Networks (GANs) (Goodfellow et al. 2014).

Classical machine learning and time-series approaches do not allow for probabilistic forecasts, or have strong assumptions on the form of the future distribution of the target variable. This leads to the inability to capture uncertainty, which is crucial for a variety of applications ranging from risk management to healthcare and self-driving cars.

To this end, we employ the ForGAN (Koochali et al. 2019) architecture, which combines GANs with recurrent neural networks (RNNs) (Medsker and Jain 2001) in order to generate probabilistic forecasts. We further introduce a novel economics-driven loss function for the generator in order to

*Corresponding author. Email: vuletic@maths.ox.ac.uk

improve Sharpe Ratio performance. In particular, we incorporate extra terms into the generator loss function, placing it into a *supervised learning* setting, and rendering it more suitable for *classification tasks* in financial applications. We denote by Fin-GANs the GANs utilising the ForGAN architecture and trained via the novel economics-driven loss function.

We compare our approach with a classical deep learning method for time-series forecasting, LSTM (Hochreiter and Schmidhuber 1997), and with a standard time-series model used in econometrics, ARIMA (Tsay 2005). We further benchmark our results against long-only strategies and against the same ForGAN architecture trained via the binary cross-entropy (BCE) loss. Additionally, we investigate the effect of the economics-driven loss function terms on the LSTM, denoting the approach by LSTM-Fin. To our best knowledge, no prior LSTM-based approaches have been trained via PnL-inspired loss functions.

An extensive set of numerical results demonstrate that our proposed methodology attains superior performance on daily stock ETF-excess returns and on daily ETF raw returns, when compared to ARIMA, LSTM, LSTM-Fin, and ForGAN trained using the BCE loss. The aforementioned loss function furthermore shifts the generated data distributions, performing classification and providing uncertainty estimates on the direction of the movement. In addition, the Fin-GAN loss helps alleviate mode collapse, which GANs are notoriously known to suffer from.

Our approach allows for scalability, and potentially modelling of the joint co-movements of different stocks. We explore the notion of universality, in the spirit of Sirignano and Cont (2019). We consider 22 different stocks, across different sectors, and 9 sector ETFs, with each industry sector having at least two different stocks included in the training data. We train the networks on the data set created by pooling together the data on all 31 tickers. We perform a similar experiment using stocks that belong to a single sector (XLP). We test our models both on the stocks included in the training set, and on stocks not seen by the model during the training stage. We find that even for the latter category of new stocks, it is possible to achieve significant Sharpe Ratios.

Summary of main contributions. We place Generative Adversarial Networks into a supervised learning setting by introducing a novel economics-based loss function for the generator. Our approach, denoted as Fin-GAN, outperforms a suite of benchmarks on daily stock market data in terms of Sharpe Ratios achieved, while producing distributional forecasts and uncertainty estimates. Furthermore, Fin-GAN allows for learning from co-movements of assets, and alleviates issues around mode collapse.

Paper outline. The remainder of the paper is organised as follows. We first survey related works in Section 2. We cover the main theory pertaining to GANs in Section 3. Section 4 reviews different performance metrics in the financial context, providing the motivation for the new loss function. Subsequently, we define the Fin-GAN loss function in Section 5, and discuss data and implementation considerations in Section 6. We detail the baseline methods in Section 7, before moving towards numerical experiments on one-step-ahead forecasting of daily stock excess returns and daily ETF returns in Section 8. Finally, Section 9 is a summary and future work outlook.

2. Related literature

Time series modelling has long been of interest in both industry and academia. A good overview of classical time-series modelling approaches can be found in Box et al. (2015). In recent years, the rise of popularity and interest in machine learning unsurprisingly resulted in an increasing number of applications to time series. However, employing machine learning for time-series modelling in financial settings has been long studied (Refenes et al. 1994).

There is a plethora of literature on the use of deep learning in finance. Guresen et al. (2011) use neural networks for stock market forecasting, while Sirignano and Cont (2019) utilise LSTMs to argue for the universality of features of asset prices. An area of particular interest for deep learning

applications is limit order book (LOB) modelling. [Sirignano \(2019\)](#) introduces spatial neural networks, aiming to model the joint distribution of future states of limit order books conditional on their current states. [Tsantekidis et al. \(2017a\)](#) utilise Convolutional Neural Networks for stock price movement forecasting in high-frequency, large-scale data extracted from LOBs, while [Tsantekidis et al. \(2017b\)](#) focus on the same problem using a Recurrent Neural Network approach. [TransLOB Wallbridge \(2020\)](#) and [DeepLOB Zhang et al. \(2019b\)](#) introduce more complex architectures with high prediction accuracy. [Zhang and Zohren \(2021\)](#) move beyond short-interval LOB forecasting, and focus on multi-horizon settings. [Lucchese et al. \(2022\)](#) introduce deepVOL, with the aim of forecasting mid-prices using a volume representation of the LOB. Deep Hedging ([Buehler et al. 2019](#)) has also sparked interest in deep learning use for hedging in both industry and academia.

GANs ([Goodfellow et al. 2014](#)) have so far been used for portfolio analysis ([Mariani et al. 2019](#)), high-frequency stock market prediction ([Zhou et al. 2018](#)), fine-tuning and calibration of financial trading strategies ([Koshiyama et al. 2020](#)), and more. QuantGAN ([Wiese et al. 2020](#)), StockGAN ([Assefa et al. 2020](#)), FINGAN ([Takahashi et al. 2019](#)) and [Buehler et al. \(2020\)](#) focus on generating synthetic financial time series. [Li et al. \(2020\)](#) and [Prenzel et al. \(2022\)](#) both employ GANs for order flow dynamics in LOBs. GANs have also been used for anomaly (manipulation) detection in financial markets ([Leangarun et al. 2018](#)), and stock market prediction via GANs has been explored and compared to other methods ([Zhang et al. 2019a](#), [Romero 2018](#)).

When it comes to GANs adapted to time series, [Yoon et al. \(2019\)](#) introduce TimeGAN and [Xu et al. \(2020\)](#) propose COT-GAN. Further to the generator and discriminator, TimeGAN utilises embedding and recovery networks in order to map the data to and from a high-dimensional latent space. COT-GAN introduces a customised loss function based on a regularised Sinkhorn distance, stemming from causal optimal transport theory. However, neither TimeGAN nor COT-GAN are tailored for a forecasting task, and are mainly used for data augmentation. Our methodology has the potential to enhance both approaches, as TimeGAN and COT-GAN could be trained by incorporating the Fin-GAN loss, which we introduce in Section 5, adapting them to the task of probabilistic forecasting of (excess) returns.

We focus on the work of [Koochali et al. \(2019\)](#), who introduced ForGAN, a conditional GAN suitable for a probabilistic time series forecasting task. This method allows for uncertainty estimates, which the classical time-series forecasting methods are unable to provide.

Another approach to achieve the same end-result relies on Gaussian Processes ([Roberts et al. 2013](#)), which are, however, very computationally expensive, and cannot capture any forecast distributions other than Gaussian. Hence, we focus on the ForGAN architecture, and in order to improve the Sharpe Ratio performance, we introduce a novel loss function for the generator. Training via customised loss functions is not uncommon in the GAN literature. For example, [Cont et al. \(2022\)](#) propose a value function suitable for estimating portfolio tail risks, [Bhatia et al. \(2021\)](#) include an extra term to the generator loss in order to generate extreme samples, and [Cont and Vuletić \(2022\)](#) incorporate a smoothness penalty for generating smooth arbitrage-free implied volatility surfaces.

Machine learning models in the univariate setting are able to successfully capture interactions between the input features and the target, even in a nonlinear setting. As previously mentioned, the most popular approaches include deep learning ([Goodfellow et al. 2016](#), [Wu et al. 2020](#)), and tree-based methods such as GBRT ([Chen and Guestrin 2016](#), [Ke et al. 2017](#), [Dorogush et al. 2018](#)). However, being able to model nonlinear *cross-asset* interactions has been a task of significant interest in the recent literature ([Wu et al. 2023, 2022](#)). The majority of such cross-asset models are mostly linear in nature and come with theoretical guarantees, but cannot effectively model the nonlinear feature-target interaction. A GAN-based approach also has the potential to capture cross-asset interactions in order to improve the overall forecasting task.

3. Generative Adversarial Networks

Generative Adversarial Networks (GANs) were introduced by Goodfellow et al. (2014) and are currently state-of-the-art for various tasks, most notably image and video generation. They also have various applications to science (Mustafa et al. 2019), video games (Wang et al. 2019), photo editing (Zhang et al. 2020), video quality enhancement (Galteri et al. 2019), and many other areas. They belong to the family of generative models and their task is generating new samples coming from the unknown data distribution.

3.1. Classical GAN

A classical way to view GANs is through a game theory lens. Suppose we have two players, the generator G and the discriminator D , playing a game against each other. G is generating what it believes is a realistic-looking sample, whereas D is determining whether or not the sample given to it is coming from the data or from the generator. The goal of G is to learn to make D believe that its outputs are coming from the data, and generate samples which look as authentic as possible. On the other hand, D is trying to learn how to properly distinguish between the data samples and those coming from the generator. The generator G improves the generated samples using the feedback it receives from the discriminator D . In GANs, both the generator G and the discriminator D are differentiable functions with respect to their parameters, and they are represented by neural networks. G is given some noise as input, usually Gaussian, and returns a *generated* sample as the output. D takes either the sample generated by G , or a *real* sample coming from the data as input, and outputs the probability of its input being *real*. GANs are trained via an adversarial process which consists of simultaneously training two different models, a generator and a discriminator. In the case of neural networks, this can be efficiently achieved using backpropagation.

Suppose that we have some data $\mathbf{x}$ coming from an unknown probability distribution p_{data} , and that we wish to generate more samples from p_{data} .

DEFINITION 1 The generator $G(\mathbf{z}; \theta_g)$ is a differentiable mapping from the latent space to the data space. It is represented by a neural network, taking input noise $\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})$. The probability distribution to which the outputs of G belong is denoted as p_g .

Remark 1 Usually, $\mathbf{z}$ is a multivariate normal.

DEFINITION 2 The discriminator $D(\mathbf{x}; \theta_d)$ is a differentiable mapping from the data space to $(0, 1)$, represented by a neural network. $D(\mathbf{x})$ represents the network's estimated probability of $\mathbf{x}$ coming from p_{data} rather than from p_g .

The discriminator D is trained to maximise the probability of assigning the correct label to both samples from p_{data} and p_g . The generator G is trained to either minimise the log-probability of D correctly assigning its outputs as coming from p_g , or to maximise the log probability of the discriminator being mistaken.

The two networks have separate cost functions which they aim to minimise. Both networks are only able to update their own respective parameters in order to minimise their corresponding cost functions, although the cost function of each network depends on the parameters of the other network. They are required to keep the parameters of the other network fixed while performing optimisation. Usually, the cost function taken for the discriminator is the cross-entropy loss.

DEFINITION 3 The cost function for the discriminator is $J^{(D)}(\theta_d, \theta_g)$ is defined as

$$J^{(D)}(\theta_d, \theta_g) = -\frac{1}{2} \mathbb{E}_{p_{data}} [\log[D(\mathbf{x}; \theta_d)]] - \frac{1}{2} \mathbb{E}_{p_{\mathbf{z}}} [\log[1 - D(G(\mathbf{z}; \theta_g); \theta_d)]] . \quad (1)$$

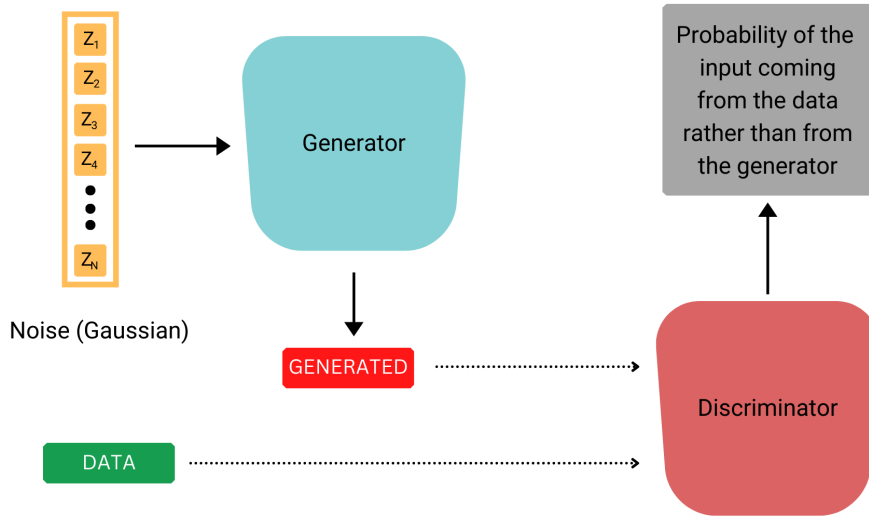


Figure 1.: An illustration of a standard GAN.

Learning the zero-sum game with this cost for the discriminator and $J^{(G)} = -J^{(D)}$ for the generator minimises the Jensen-Shannon divergence between p_{data} and p_g . The game converges to a Nash equilibrium if both policies can be updated in the function space, which is usually not the case in practice, as argued by Goodfellow (2017). When convergence takes place, the discriminator views all outputs as being equally likely to come from p_{data} or p_g , therefore having its outputs converging to $\frac{1}{2}$. However, Goodfellow et al. (2014), Goodfellow (2017) argue that maximising the cross-entropy loss instead of optimising the zero-sum game should be favoured. This approach corresponds to the generator minimising the log probability of the discriminator being mistaken. That is, the generator is trying to obtain as high of a score from the discriminator as possible.

DEFINITION 4 The cross-entropy loss for the generator is $J^{(G)}(\theta_d, \theta_g)$ is defined as

$$J^{(G)}(\theta_d, \theta_g) = -\frac{1}{2} \mathbb{E}_{p_z} [\log[D(G(z; \theta_g); \theta_d)]] . \quad (2)$$

A further argument made by Goodfellow et al. (2014) for using the cross-entropy loss for the generator, i.e. maximising $\log[D(G(z; \theta_g); \theta_d)]$, is that the zero-sum loss may not provide sufficiently large gradients for G to be able to learn well. Early in training, samples from G can be easily distinguishable as not coming from the data, and D can strongly classify them as such, making $\log[1 - D(G(z; \theta_g); \theta_d)]$ saturated. This makes it more difficult for G to learn. Changing the objective to maximising $\log[D(G(z; \theta_g); \theta_d)]$ results in the same equilibrium point of the dynamics of D and G , while allowing for gradients higher in magnitude, allowing the generator G to learn more. It is common to train the discriminator more, performing k discriminator updates for each generator update. However, Goodfellow (2017) argues that $k = 1$ should be used. An illustration of a standard GAN can be found in Figure 1.

The main problem that arises when training GANs is *mode collapse*, also known as the *Helvetica scenario*. It can be full and partial. When it happens, the generator yields the same outputs (full mode collapse) no matter the noise, or the outputs all have similar motifs (partial mode collapse). Durall et al. (2021) argues that mode collapse is related to the generator converging towards sharp local minima. When the generator starts reproducing similar outputs and the discriminator cannot identify them as stemming from the generator, due to potentially converging to local minima, the generator receives a high rating from the discriminator. This can lead to weak gradients and close to no updates to the generator network, which can ultimately result in mode collapse.

It is worth noting that keeping the generator and the discriminator in balance is of utmost importance. This is particularly true when the discriminator overpowers the generator. When the

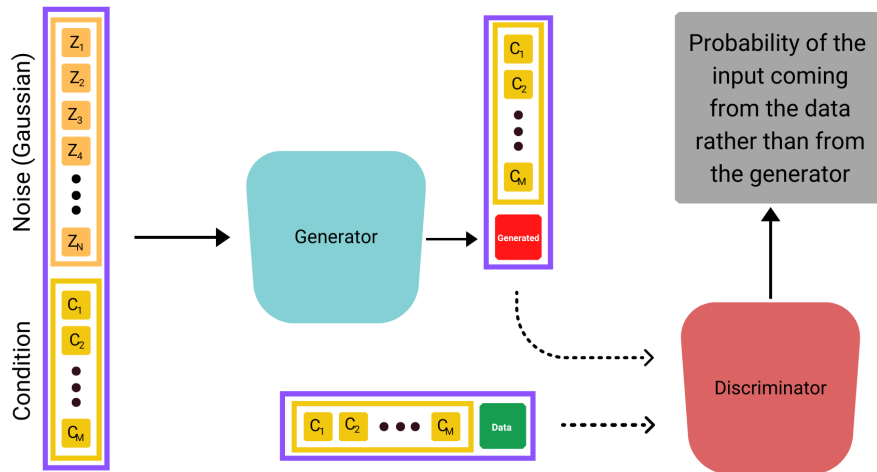


Figure 2.: An illustration of a conditional GAN pipeline.

discriminator is too successful, the generator cannot learn much from the feedback it receives, due to weak gradients.

In general, GANs are very difficult to train (Arjovsky and Bottou 2017), hence it is natural to consider changing the training objective. Various objectives have been shown to perform well (Huszár 2015), including all f-divergences (Nowozin et al. 2016). However, the most commonly used GAN variation, which is not the standard cross-entropy or zero-sum GAN, is a Wasserstein GAN (W-GAN), introduced by Arjovsky et al. (2017). It minimises the Wasserstein-1 distance between p_g and p_{data} . Usually, Wasserstein GANs with Gradient Penalty (Gulrajani et al. 2017) are preferred, as they softly enforce the Lipschitz constraint. This approach includes a gradient penalty to the training objective of the discriminator.

3.2. Conditional GAN

Conditional generative adversarial networks are a natural extension to standard GANs and were first mentioned in the initial GAN paper (Goodfellow et al. 2014), and first explored by Mirza and Osindero (2014). Instead of having all data samples from the same underlying distribution, we assume that the data $\mathbf{x}$ is coming from a conditional distribution $p_{data}(\mathbf{x}|\text{condition})$, given some deterministic condition. The same principle applies in this case. The only difference in the architecture is that both the generator and the discriminator are given further deterministic information (condition) alongside the standard inputs. When this condition is a label, it is usually encoded as a one-hot-vector. The discriminator/critic is also given the same condition. An illustration of a conditional GAN can be found in Figure 2.

As an intuitive illustrative example, we may aim to generate photos of cats and dogs. We have available labels for cats and dogs given as one-hot vectors. The generator would receive noise and a label (say for cats) as an input, and output what it thinks is a photo of a cat. The discriminator would take as input a photo of a cat from the original data, as well as the one generated by the generator network, both alongside the label indicating that a photo should be that of a cat. The discriminator would then give feedback on both the cat photo coming from the real data, and the one coming from the generator, and learn.

3.3. ForGAN

ForGAN is a conditional GAN whose architecture is well-suited for probabilistic forecasting of time series, and was introduced by Koochali et al. (2019). The idea is to use a conditional GAN to make

a probabilistic forecast of x_{t+1} given the previous L values of the series, $x_t, x_{t-1}, \dots, x_{t-(L-1)}$, which we denote by $\mathbf{x}_{t+1}^{-L}$. This allows for much more information to be leveraged, compared to standard point estimates, and renders the approach amenable to obtaining uncertainty estimates.

In order to exploit the time-series structure, both the generator and the discriminator first pass their inputs through a recurrent neural network layer. The ForGAN architecture includes either a gated recurrent unit (GRU) (Cho et al. 2014) or a long short-term memory (LSTM) cell (Hochreiter and Schmidhuber 1997), depending on the data. In our work, we chose to work with the LSTM cell because of its prevalence in finance. Similar architectures have already been successfully used for stock market prediction tasks, for example, Zhang et al. (2019a). Diagrams illustrating the ForGAN architecture using LSTM cells are shown in Figure 3.

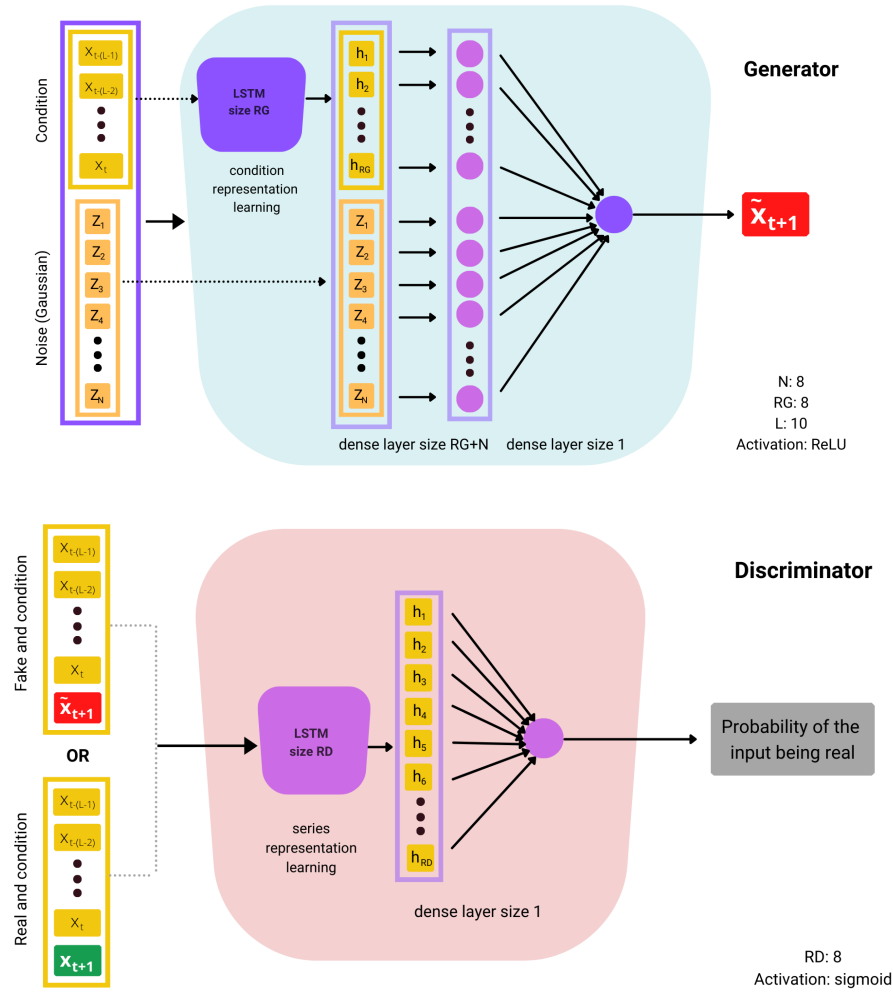


Figure 3.: ForGAN architecture using an LSTM cell.

4. Performance measures

There are a number of ways to measure model performance. Based on applications, not all performance metrics are equally important. In order to compare one approach to another, it is crucial to understand which properties of the data we care about the most, in order to be able to construct appropriate performance metrics.

Standard approach for GANs. Measuring the performance of GANs is very difficult as we often do not know what distribution the real data is coming from. Usually, when working with images, videos, or sound, a number of human annotators are there to give feedback on both the generated and real samples. Alternatively, *inception score* (Salimans et al. 2016) can be used. It is based on the Kullback-Leibler divergence between the conditional data distribution given the label and the marginal label distribution. This score also takes into account the diversity of generated samples. Unfortunately, none of the approaches mentioned above can be applied to our setting.

Standard approach for time series. Traditionally, when the task at hand is time series forecasting, one opts for performance metrics such as MSE (mean squared error), RMSE (root mean squared error), and MAE (mean absolute error). To define MAE and RMSE, suppose that the real value we wish to estimate is $\mathbf{x}$ and that our forecast is $\hat{\mathbf{x}}$. This can be one GAN output given a single noise sample, or the mean or the mode of the generated distribution given the corresponding condition. Then, the mean absolute error and the root mean squared error are defined as

$$MAE(\mathbf{x}, \hat{\mathbf{x}}) = \frac{1}{n} \sum_{i=1}^n |x_i - \hat{x}_i|, \quad RMSE(\mathbf{x}, \hat{\mathbf{x}}) = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2}. \quad (3)$$

Financial data, forecasting returns. As we are working with financial data, MAE and RMSE are not as representative as they are in traditional time-series settings. Forecasting in a financial context is oftentimes performed for the purpose of determining the side of a trade (i.e., whether to buy or sell an asset), which provides motivation for considering a metric based on profit and loss (PnL). While both MAE and RMSE can be very low, it could still be possible to misclassify the side/directionality of a large move in the underlying asset price, leading to a severe PnL loss. Hence we incorporate the PnL as one of our main performance metrics. At each point in time, we produce a forecast of the true value of the (ETF-excess) returns time series, and place a trade according to our forecast. If our forecast of ETF-excess returns is positive, we long the stock and short the corresponding ETF. Conversely, if it is negative, we place the opposite trade. This sequence of trades results in a time series of PnLs. To minimise risk and maximise profit, one aims for the PnL time series to be positive, and have low variance. The larger the average of the PnL series is, the higher the profitability of our strategy. The lower the variance of the PnL series, the lower the risk associated with the strategy. Altogether, this leads us to employ the *annualised Sharpe Ratio* as our main performance measure. We remark that this is also the metric commonly used by portfolio managers to assess the risk-adjusted performance of their strategies.

Next, we define the above terms in the context of the data we are using. When considering stocks, we work with the market-excess returns, while for ETFs, we use raw returns. Assuming that asset A has price S_t^A at time t , its return at time $t + 1$ is defined as

$$r_{t+1}^A = \log \left(\frac{S_{t+1}^A}{S_t^A} \right). \quad (4)$$

If the sector to which asset A belongs, has the corresponding ETF I whose return at time t is r_t^I , then the excess return of the asset A at time t is defined as

$$re_t^A = r_t^A - r_t^I. \quad (5)$$

If we expect A to have a higher return than I , then we long one share of stock A and short one share of ETF I . We do analogously for the opposite case. Hence, if our forecast of re_{t+1}^A is $\hat{re}_{t+1}^A$, the PnL of this strategy (at time $t + 1$) is

$$PnL_{t+1} = \text{sign}(\hat{re}_{t+1}^A) re_{t+1}^A, \quad (6)$$

where we use the left-continuous version of the sign function

$$\text{sign}(x) = \begin{cases} 1 & \text{if } x \geq 0, \\ -1 & \text{if } x < 0. \end{cases} \quad (7)$$

To streamline the analysis and comparisons, we consider the PnL in basis points and normalise by dividing by the number of total trades placed. In our setting, the number of trades is the size of the validation/test set. If there are n values of re^A that we aim to forecast, denoted $re_1^A, \dots, re_n^A$, we consider the PnL per trade (in basis points), and for simplicity refer to it as the $pPnL$ henceforth. Formally, this amounts to

$$pPnL = \frac{10000}{n} \sum_{i=1}^n \text{sign}(\hat{re}_i^A) re_i^A. \quad (8)$$

We note that the $pPnL$ defined by (8) does not take into account the uncertainty about the sign of the forecast, and only considers point estimates.

As we have access to full probability forecasts, we would ideally place larger weights on the returns for which we have higher confidence, and lower on those for which we have lower certainty about. Hence, we estimate the probability of the (ETF-excess) return forecast being positive and negative by taking $B = 1000$ different noise samples for the same condition. The two probability estimates are denoted by p_u and p_d , and are more precisely defined as

$$p_u^i = \frac{1}{B} \sum_{j=1}^B \mathbb{1}_{\hat{re}_i^A(\mathbf{z}^j) \geq 0}, \quad p_d^i = \frac{1}{B} \sum_{j=1}^B \mathbb{1}_{\hat{re}_i^A(\mathbf{z}^j) < 0}, \quad (9)$$

where $\hat{re}_i^A(\mathbf{z}^j)$ is the generator's output given the previous L values of the time-series $\mathbf{re}_i^{A-L}$ and noise $\mathbf{z}^j$. We can now define an alternative PnL, which we can construe as the PnL of a weighted strategy which takes into account our certainty of the sign of the forecast, as

$$wPnL_2^i = 10000(p_u^i - p_d^i)re_i^A. \quad (10)$$

This strategy corresponds to going p_u^i long stock and p_u^i short ETF, and p_d^i short stock and p_d^i long ETF. This is the PnL of an equivalent strategy, consisting of taking an average bet of each of the generator's outcomes given appropriate conditions across different noise samples. Hence, we are penalising higher uncertainty more, and if we are not very certain about the outcome, we attain significantly lower potential loss.

Note that we are multiplying the PnL by 10,000, in order to obtain the PnL value in basis points. Furthermore, we consider daily data, for which we have available two sets of returns per day (open-to-close and close-to-open), which motivates us to consider the daily PnL

$$wPnL^i = wPnL_2^{2i} + wPnL_2^{2i+1}, \quad i = 1, \dots, \lfloor \frac{n}{2} \rfloor. \quad (11)$$

The more symmetric distributions we learn, the closer $wPnL$ is to zero. We use the weighted strategy introduced above to compute the annualised Sharpe Ratio, which we use as our main performance metric

$$SR_w = \frac{PnL_w}{\left(\frac{1}{\lfloor \frac{n}{2} \rfloor} \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} (wPnL^i - PnL_w)^2 \right)^{1/2}}, \quad PnL_w = \frac{1}{\lfloor \frac{n}{2} \rfloor} \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} wPnL^i. \quad (12)$$

When working with models providing point estimates only, we are unable to employ the weighted strategy, hence we instead consider

$$PnL^i = 10000 (\text{sign}(\hat{r}e_{2i}^A)re_{2i}^A + \text{sign}(\hat{r}e_{2i+1}^A)re_{2i+1}^A), \quad i = 1, \dots, \lfloor \frac{n}{2} \rfloor, \quad (13)$$

and the corresponding annualised Sharpe Ratio

$$SR = \sqrt{252} \frac{PnL_m}{\left(\frac{1}{\lfloor \frac{n}{2} \rfloor} \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} (PnL^i - PnL_m)^2 \right)^{1/2}}, \quad PnL_m = \frac{1}{\lfloor \frac{n}{2} \rfloor} \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} PnL^i. \quad (14)$$

Annualised Sharpe Ratios of above 1 are considered to be *good*, above 2 are considered to be *very good* and those above 3 are referred to as *excellent*. When reporting the PnL, we use the mean daily PnL, that is PnL_w , or PnL_m , depending on the setting.

Remark 2 Note that we do not take transaction costs into account.

5. Fin-GAN loss function

The main contribution of our work lies in introducing a novel economics-driven loss function for the generator, which places GANs into a **supervised learning** setting. This approach allows us to move beyond traditionally employed methods for pointwise forecasting and move towards probabilistic forecasts, which offer uncertainty estimates. Furthermore, we are evaluating the performance based on the sign of the predictions, meaning that we are interested in **classification**, which our loss function is more suitable for.

5.1. Motivation

The main aim of GANs is to create synthetic samples mimicking real data. However, an important question to ask is which properties of the real data would we like to reproduce via simulations. In our case, it is the sign of the forecast. We are more interested in correctly classifying returns/excess returns, especially when the magnitude of the price moves is large, rather than in producing forecasts close to the realised value. The motivation is that one may produce a forecast which is close to the realised value of the time series, but has the opposite sign. In financial time series forecasting, *correctly estimating the sign when it matters the most* is often more important than forecasting close to the realised value. We aim to provide the generator with more information, such that it better replicates the sign of the data, which is the crucial component of the task. The main reasons and benefits of using the novel loss function terms can be summarised as follows:

- shift the generated conditional distributions in the correct direction,
- render GANs more amenable to a classification task,
- provide effective regularisation,
- help the generator learn more, especially in the case of weak gradients,
- help evade mode collapse; by making the generator loss surface more complex, the generator is likely to converge towards sharp local minima.

We have already seen several examples of different loss functions, along with the benefits they bring. Most notably, the W-GAN-GP (Gulrajani et al. 2017) adds an extra term to the discriminator loss as a way to enforce the Lipschitz constraint. Adding loss terms to the generator is less common, but it was employed in the recent work by Bhatia et al. (2021) for generating extreme samples, and in VOLGAN (Cont and Vuletić 2022) for simulating arbitrage-free implied volatility surfaces.

Other customised loss functions have also shown to be useful in finance, such as Tail-GAN (Cont et al. 2022), which focuses on the estimation of portfolio tail risks.

5.2. Novel loss function

Assume that our GAN aims to forecast x_i , where $i = 1, \dots, n_{batch}$. For each x_i , given a noise sample $\mathbf{z}$ and the condition consisting of the previous L values of the time-series $\mathbf{x}_i^{-L}$, the output of the generator is

$$\hat{x}_i = G(\mathbf{z}, \mathbf{x}_i^{-L}). \quad (15)$$

Let $\mathbf{x} = (x_1, \dots, x_{n_{batch}})$ and $\hat{\mathbf{x}} = (\hat{x}_1, \dots, \hat{x}_{n_{batch}})$ be the forecast of $\mathbf{x}$. We define three loss function terms, namely PnL, MSE, and Sharpe Ratio based, which we use to train the generator.

PnL term. The first and most obvious choice of a loss function term to included in the generator's training objective (to maximise) is the PnL. However, the sign function is not differentiable, rendering it impossible to perform backpropagation. In order to alleviate this issue, we propose a smooth approximation to the pPnL, which we denote as PnL^* , defined as

$$PnL^*(\mathbf{x}, \hat{\mathbf{x}}) = \frac{1}{n_{batch}} \sum_{i=1}^{n_{batch}} PnL_a^*(x_i, \hat{x}_i), \quad (16)$$

where PnL_a^* is a smooth approximation to the PnL for a particular forecast

$$PnL_a^*(x_i, \hat{x}_i) = \tanh(k_{tanh}\hat{x}_i)x_i, \quad (17)$$

and $\tanh(x)$ is defined by

$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}. \quad (18)$$

The hyperparameter k_{tanh} controls the accuracy of our approximation. As values of excess returns are usually small, it is desirable for k_{tanh} to be large enough to have a good approximation, but at the same time small enough to have strong gradients which the generator can learn from. Note that in the expression for the PnL term, we consider the PnL per trade. This is not the PnL averaged over days as the training data set is shuffled at the start of every epoch. We use $k_{tanh} = 100$.

MSE term. Even though we are mainly interested in the sign of our forecast, we would also like to have $\hat{x}_i$ close to x_i . In order to enforce this, we add an MSE term to the training objective

$$MSE(\mathbf{x}, \hat{\mathbf{x}}) = \frac{1}{n_{batch}} \sum_{i=1}^{n_{batch}} (x_i - \hat{x}_i)^2. \quad (19)$$

Sharpe Ratio term. As our main measure of performance is the annualised Sharpe Ratio, we introduce an SR-based loss function term for the generator to maximise. However, due to the prohibitive computational cost, we did not implement the weighted strategy during training, and instead worked with point estimates, given one noise sample per condition window. We do not pair up the trades in two, as the consecutive data points in the training set do not necessarily come in that order originally due to shuffling. It is very important to pick mini-batches randomly, or equivalently to shuffle the training data set at the start of every epoch, in order for the stochastic

gradient descent to converge. We hence define the Sharpe Ratio term to be maximised by the generator during training, denoted by SR^*

$$SR^*(\mathbf{x}, \hat{\mathbf{x}}) = \frac{\frac{1}{n_{batch}} \sum_{i=1}^{n_{batch}} PnL_a^*(x_i, \hat{x}_i)}{\sqrt{\frac{1}{n_{batch}} \sum_{i=1}^{n_{batch}} \left(PnL_a^*(x_i, \hat{x}_i) - \frac{1}{n_{batch}} \sum_{j=1}^{n_{batch}} PnL_a^*(x_i, \hat{x}_i) \right)^2}}. \quad (20)$$

Standard deviation (of the PnL series) term. Maximising the Sharpe Ratio means maximising the PnL and minimising the standard deviation of the PnLs. Hence, it is natural to consider a standard deviation term, which we define as

$$STD(\mathbf{x}, \hat{\mathbf{x}}) = \sqrt{\frac{1}{n_{batch}} \sum_{i=1}^{n_{batch}} \left(PnL_a^*(x_i, \hat{x}_i) - \frac{1}{n_{batch}} \sum_{j=1}^{n_{batch}} PnL_a^*(x_i, \hat{x}_i) \right)^2}. \quad (21)$$

As the loss term defined above refers to the standard deviation of approximate PnLs, it should only be included in the training objective if the PnL^* term is included, due to the hierarchy principle. The SR^* term and the PnL^* combined with the STD term convey the same information, so the three should not be utilised at the same time. However, although the goal of maximising SR^* is the same of simultaneously maximising PnL^* and minimising STD , the gradient norms are different, which may result in different behaviour during training.

An economics-driven loss function for the generator. Finally, we integrate all of the aforementioned loss function terms defined in Equations (16), (19) and (20), and arrive at the Fin-GAN loss for the generator

$$L^G(\mathbf{x}, \hat{\mathbf{x}}) = J^{(G)}(\mathbf{x}, \hat{\mathbf{x}}) - \alpha PnL^*(\mathbf{x}, \hat{\mathbf{x}}) + \beta MSE(\mathbf{x}, \hat{\mathbf{x}}) - \gamma SR^*(\mathbf{x}, \hat{\mathbf{x}}) + \delta STD(\mathbf{x}, \hat{\mathbf{x}}), \quad (22)$$

where $J^{(G)}$ is one of the standard GAN losses (zero-sum, cross-entropy or Wasserstein), and the hyperparameters α, β, γ are non-negative. The loss for the discriminator remains $J^{(D)}$. In prior numerical experiments, the Wasserstein loss suffered from explosion, hence we opted to use the binary cross entropy loss for $J^{(G)}$. We aim to minimise a classical generator loss function ($J^{(G)}$), maximise the PnL-based loss function term, minimise the MSE term, and maximise the SR-based loss function term, or jointly maximise the PnL and minimise the STD term. The loss function combinations which we investigate are PnL^* , $PnL^* \& STD$, $PnL^* \& MSE$, $PnL^* \& SR^*$, $PnL^* \& MSE \& STD$, $PnL^* \& MSE \& SR^*$, SR^* , $SR^* \& MSE$, MSE , BCE only. The standard deviation term refers to the PnL term, so due to the hierarchy principle, it is included only if the PnL term is. When it comes to Fin-GAN, there needs to be an economics-driven loss term included, hence for Fin-GAN we only consider the following options: PnL^* , $PnL^* \& STD$, $PnL^* \& MSE$, $PnL^* \& SR^*$, $PnL^* \& MSE \& STD$, $PnL^* \& MSE \& SR^*$, SR^* , $SR^* \& MSE$. In other words, we impose the following conditions on $\alpha, \beta, \gamma, \delta$:

- $\max(\alpha, \gamma, \delta) > 0$ (at least one additional term other than MSE is included).
- If $\beta > 0$ then $\max(\alpha, \gamma, \delta) > 0$ (if the MSE term is included, then another term is included).
- If $\delta > 0$, then $\alpha > 0$ (the STD term is included only if the PnL^* term is).
- $\min(\gamma, \delta) = 0$ (SR^* and STD terms are never included at the same time).

We refer to GANs utilising the ForGAN architecture and the loss function defined by Equation (22) and the rules above as Fin-GANs.

Gradient norm "matching". In order to avoid hyperparameter tuning to find $\alpha, \beta, \gamma, \delta$, we use approach similar to that of Cont and Vuletić (2022), treating all terms as *equally important*. That

is, we first train the networks using $J^{(G)}$ only, but we calculate the norms of gradients with respect to all terms in (22). We then set $\alpha, \beta, \gamma, \delta$ to be the means of observed ratios (or zero if they are not included in the objective) of the gradient norms with respect to the BCE term and the gradient norms with respect to the corresponding loss function terms. We then train the networks using the possible combinations with the calculated values of $\alpha, \beta, \gamma, \delta$. The final combination of the novel terms is then determined by evaluating the Sharpe Ratio on the validation set. Further training details are explained in Section 6, and the Fin-GAN algorithm is given in Algorithm 1.

5.3. Benefits and challenges of the Fin-GAN loss function

The novel loss function terms do indeed shift the generated distributions, help evade mode collapse, and improve Sharpe Ratio performance, which we demonstrate in an extensive set of numerical experiments. Including the new loss function terms introduces four new hyperparameters to be tuned, rendering the optimisation a more challenging problem. However, the *gradient norm matching* approach mitigates this issue.

In addition, the loss surface becomes more complex, raising convergence challenges. In initial numerical experiments, when exploring a wider range of hyperparameters α, β, γ , the MSE and the Sharpe Ratio terms appeared to encourage the generator to produce very narrow distributions. It is not immediately obvious why the Sharpe Ratio term with a high γ coefficient would result in mode collapse, as it encourages the *PnLs* to have a low standard deviation, and not the generated samples. However, including the PnL-based term helped alleviate the issue of mode collapse, which is related to convergence to sharp local minima (Durall et al. 2021). The PnL term helps the generator escape such areas of the loss surface.

When *gradient norm matching* was used to tune the hyperparameters $\alpha, \beta, \gamma, \delta$, there was no mode collapse in Fin-GAN no matter the initialisation of network weights. However, we noticed that using Xavier initialisation (Glorot and Bengio 2010) instead of He initialisation (He et al. 2015) helped reduce the number of iterations of ForGAN with mode collapse. With He initialisation, this was occurring in 67% of the cases. Utilising normal Xavier initialisation resolved this issue.

The classical generator loss term, i.e. the feedback received from the discriminator, allows learning conditional probability distributions, instead of only pointwise forecasts. The remaining loss function terms promote sign consistency (enforced by the PnL and Sharpe Ratio terms), while at the same time targeting to remain close to the realised value (MSE).

If the hyperparameters $\alpha, \beta, \gamma, \delta$ are too large, the feedback of the discriminator becomes negligible, and we move towards the standard supervised learning setting, using a generator-only model. On the other hand, if $\alpha, \beta, \gamma, \delta$ are too small, we are not providing any extra information to the generator, thus remaining in a classical GAN setting. The additional information about the data communicated through the novel loss function terms is especially invaluable in the case of weak gradients coming from the discriminator/critic feedback, as the generator can still make progress in this case. This all together leads to a better classification of the data in terms of directional price movements, i.e. correctly classifying the sign of future time series values, alongside uncertainty estimates of the movement direction. The gradient norm matching procedure ensures that all loss terms are treated as equally important, and that the corresponding hyperparameters are neither too large, nor too small.

6. Data description and implementation considerations

In this section, we outline the main characteristics of the data sets we use, and further describe the ForGAN architecture. Next, we discuss the training setup, including the hyperparameter choice as well as optimisation. Finally, we demonstrate the general behaviour of the Fin-GAN model, which recurrently surfaced throughout the numerical experiments.

Sector name	ETF ticker	Stock tickers
Consumer Discretionary	XLY	AMZN, HD, NKE
Consumer	XLP	CL, EL, KO, PEP
Energy	XLE	APA, OXY
Financial	XLF	WFC, GS, BLK
Health Care	XLV	PFE, HUM
Industrial	XLI	FDX, GD
Technology	XKL	IBM, TER
Materials	XLB	ECL, IP
Utilities	XLU	DTE, WEC

Table 1.: Tickers and ETFs used for numerical experiments.

6.1. Data description

We consider daily stock ETF-excess returns and daily raw ETF returns, extracted from CRSP on [Wharton Research Data Services](#). The time frame of interest is Jan 2000-Dec 2021. All time series have the same training-validation-testing split, namely 80 – 10 – 10. That is, the training period is 3rd Jan 2000 - 9th Aug 2017, the validation period is 10th Aug 2017 - 22nd Oct 2019, and the testing period is 23rd Oct 2019 - 31st Dec 2021. These three time periods are very different from an economic perspective, and it is important to note that the test data includes the start of the Covid-19 pandemic, rendering the problem more challenging. We consider close-to-open and open-to-close returns sequentially, and refer to each one of them as one unit of time. In other words, one input time series alternates intraday open-to-close and overnight close-to-open returns. The sliding window is one time unit, the condition window is ten time units (corresponding to five days of returns, i.e. one full trading week), and the prediction window is one time unit ahead.

The prices are adjusted by division with the cumulative adjustment factor provided (*cfacpr*), and the returns are capped at $\pm 15\%$, to alleviate the effect of potential outliers. To pre-process the data, we first create one time series consisting of consecutive close-to-open and open-to-close returns, then chronologically split the data into three disjoint shorter time series (training-validation-testing), and we appropriately further process the three resulting time series. At this stage, we create a matrix for each of the three data sets; each such matrix containing 11 columns, the first 10 corresponding to the condition window ($x_1, \dots, x_{10}$), and the last column being the target (x_{11}). A sliding window of one time unit implies that the k^{th} value in the column j is the $k + 1^{st}$ value in the column $j - 1$.

We consider ETF-excess returns for 22 different stocks across nine sectors, and **raw returns** of the nine sector ETFs. All stocks are current members of the S&P500 index. The tickers, along with their sector memberships, are shown in Table 1.

6.2. ForGAN Architecture

For the single-stock/ETF numerical experiments, the classical ForGAN architecture utilising LSTM cells, as shown in Figure 3, was used. Due to the small size of the data sets used, the corresponding layer dimensions and the noise dimension are all set to 8. The generator uses ReLU as activation function, while the discriminator uses the sigmoid.

6.3. Training

Avoiding further hyperparameter tuning, the optimiser used throughout is RMSProp ([Hinton et al. 2012](#)), and normal Xavier initialisation ([Glorot and Bengio 2010](#)) is used for the weights. The discriminator/critic and the generator are trained *using an alternating direction method*, having

$k = 1$ discriminator update per one generator update. We use reference batch normalisation, picking the first 100 samples from the training data serving as the reference batch, in order to not be anticipative. Mini-batch size is also set to 100. The learning rates of both networks are set to 0.0001. We train for 25 epochs at the start, using the BCE loss only in order to implement the *gradient norm matching*, and find the values for $\alpha, \beta, \gamma, \delta$ coefficients. We then *branch away* from the generator and the discriminator by training via every suitable loss combination for 100 more epochs. At the end of the training stage, we calculate Sharpe Ratio on the validation set, and choose the loss function combination, and the optimised generator which maximises it. Due to differences in economic periods for validation and testing, it may not be the case that the networks which would perform well on the test set are chosen. However, this approach allows us to ensure good performance, regardless of periods of crisis. The code is implemented in PyTorch. The Fin-GAN algorithm is summarised in Algorithm 1.

Remark 3 One may explore a wider range of hyperparameters $\alpha, \beta, \gamma, \delta$, learning rates lr_g, lr_d , hidden dimensions RG, RD , noise dimension N , as well as completely different architectures, by evaluating Sharpe Ratio on the validation set, and opting for the hyperparameter/architecture maximising it.

6.4. Gradient stability

Before incorporating the additional loss function terms, we first investigate the behaviour of the gradient norms during training, in order to check for exploding and vanishing gradients. We train ForGAN, performing updates on the BCE loss using RMSProp (Hinton et al. 2012), for 25 epochs, the period used for *gradient norm matching*. In Figure 4, we display sample gradient norms of each term Equation (22). The data used in this example is the daily excess stock returns time series of PFE. We observe that there are no vanishing or explosion phenomena, and that the gradient norms are on a similar scale. This also holds true for other tickers used in our experiments.

6.5. Mode collapse

As we are considering time series data, we identify two different types of mode collapse, *distribution mode collapse* and *mean mode collapse*, each of which can be full or partial.

- *Distribution mode collapse* refers to a collapse in the generated distribution given a condition,
- *Mean mode collapse* refers to a collapse in the distribution of the means of generated distributions across all conditions.

It is a major concern when both full mean and full distribution mode collapse occur simultaneously, as this leads to only having a single point-estimate. In order to simplify the notion of mode collapse in our setting, we consider that mode collapse occurs if the out-of-sample generated scenarios for a particular condition (time point), that is the time series of the generated means, have standard deviation below a threshold ϵ , which we set to 0.0002. Using Xavier initialisation for the network weights (Glorot and Bengio 2010) helped alleviate the mode collapse of ForGAN. The additional terms of the Fin-GAN loss function have shown to have effective regularisation, as there was no mode collapse in any of the experiments, no matter if the networks were initialised according to Xavier or He initialisation (He et al. 2015). It is important to note that ForGAN suffered from either type of mode collapse in 67% of simulations when He initialisation was used. Furthermore, in the initial numerical experiments (prior to considering the STD term), in which a wider range of hyperparameters was used instead of performing *gradient matching*, we noticed that the MSE and SR terms with high β and γ were likely to cause mode collapse. However, the inclusion of the PnL term would resolve this issue. The consequence of the high hyperparameter next to the MSE term resulting in mode collapse is not surprising as the MSE term is encouraging the generated

Algorithm 1: Fin-GAN algorithm

Input: Hidden dimension of the generator $hid_g = 8$, hidden dimension of the discriminator $hid_d = 8$, noise dimension $noise_{dim} = 8$. Target size $pred = 1$, condition window $h = 10$, sliding window $l = 1$. Discriminator learning rate $lr_d = 0.0001$, generator learning rate $lr_g = 0.0001$. Training data, validation data, testing data. Number of epochs for gradient matching $n_{grad} = 25$, number of epochs for training $n_{epochs} = 100$, minibatch size $n_{batch} = 100$, mode collapse threshold $\epsilon = 0.0002$. Number of samples for the weighted strategy $B = 1000$.

Step 0: Take the first n_{batch} items from the training dataset as the reference batch for normalisation. Initialise the generator gen and the discriminator $disc$ networks.

Step 1: *Matching the gradient norms to find $\alpha, \beta, \gamma, \delta$.*

- Optimise gen and $disc$ using only the BCE objective for n_{grad} epochs.
- Calculate gradient norms with respect to the BCE , PnL^* , MSE , SR^* , STD terms at each optimisation update step.
- Set $\alpha, \beta, \gamma, \delta$ to be the means of observed ratios of the gradient norms with respect to the BCE term and the gradient norms with respect to the PnL^* , MSE , SR^* , STD terms, respectively.

Step 2: *Training stage.*

- Label the possible loss function combinations (PnL^* , $PnL^* \& STD$, $PnL^* \& MSE$, $PnL^* \& SR^*$, $PnL^* \& MSE \& STD$, $PnL^* \& MSE \& SR^*$, SR^* , $SR^* \& MSE$) as L_i , for $i = 1, \dots, 8$.
- Starting from gen and $disc$, with the previously set $\alpha, \beta, \gamma, \delta$. For $i = 1, \dots, 8$:
 - Reach $gen_i, disc_i$ by training $gen, disc$ for n_{epoch} epochs via the loss L_i .
 - Calculate the annualised Sharpe Ratio of the weighted strategy SR_w on the validation set by considering B noise samples for each condition. Label it as SR_{val}^i .
- Find the maximum SR_{val}^i , achieved by the loss L_{i^*} .

Step 3: *Evaluation on the test set.*

- Load gen_{i^*} as the trained Fin-GAN generator.
 - Consider B noise samples for each condition in the test set. As point estimates for each condition, compute the mean of the generated distribution.
 - Investigate mode collapse by considering the generated distribution for a particular condition, and the distribution of generated means. Label as mode collapse if the corresponding standard deviation is below the threshold ϵ .
 - Calculate MAE and RMSE using the point estimates (means).
 - Implement the weighted strategy using the simulated distributions. Pair up trades into two to reach one PnL per day, and calculate the annualised Sharpe Ratio SR_w and the mean daily PnL PnL_w .
-

values to be close to the target. The Sharpe Ratio term, even though promoting small standard deviation, is not promoting narrow distributions of forecasts, but PnL s.

6.6. Generated distributions

Samples of generated distributions when training using different Fin-GAN loss function term combinations are displayed in Figure 5. All of the distributions shown are generated using the same training data (excess returns of PFE), and the same condition in the test set (they have the same target). Throughout the numerical experiments, training via just the BCE loss on its own would produce more symmetrical distributions, whereas the incorporated Fin-GAN loss function terms

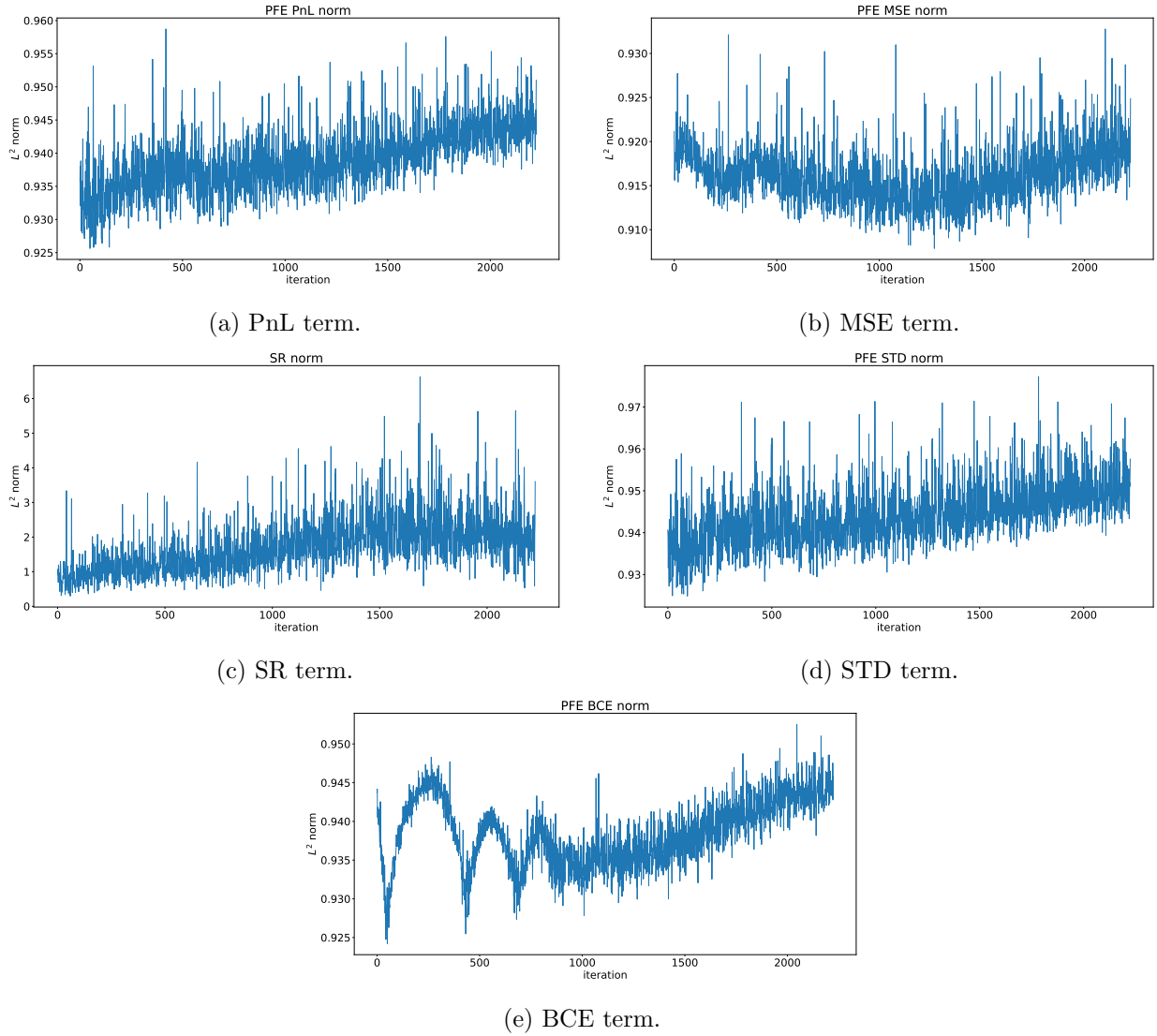


Figure 4.: Sample generator gradient norms during training with respect to different terms (PnL, MSE, SR, STD, BCE). Updates were performed using the BCE loss only.

would help shift the forecast distributions. We also investigate the generated means and compare them with the true distributions of the target in Figure 6. We note that in this particular example, the PnL and STD combination has a similar behaviour to the SR term on its own. This is not surprising as both convey the same information, although they have different gradients. Both options have the biggest distribution-shifting effect in this example. We note that apart from the PnL with STD, and SR loss terms, all of the generated means resemble the true target distribution. Furthermore, including the *MSE* term prompted the distribution of the means to be closer to that of the true returns.

Figure 5 allows us to illustrate the weighted strategy that GANs are able to employ due to uncertainty estimates. In this particular forecast, the SR, as well as PnL and STD, would result in long ETF and short stock with weight one, whereas the PnL, MSE and SR combination would have a similar trade of a slightly lower weight, and the other combinations would result in very small trades due to uncertainty about the sign of the forecast. Although we are considering the Fin-GAN loss terms only in Figures 5 and 6, it is important to note that the average correlation between the BCE loss and the BCE loss with the MSE term added to it is 99.7%.

We observe that it is beneficial to train on a combination of the loss terms, as they are able to

produce shifts in generated distributions and evade mode collapse. The benefit of jointly utilising the Fin-GAN loss function terms is backed up by the Sharpe Ratio performance, which we analyse in Section 8.

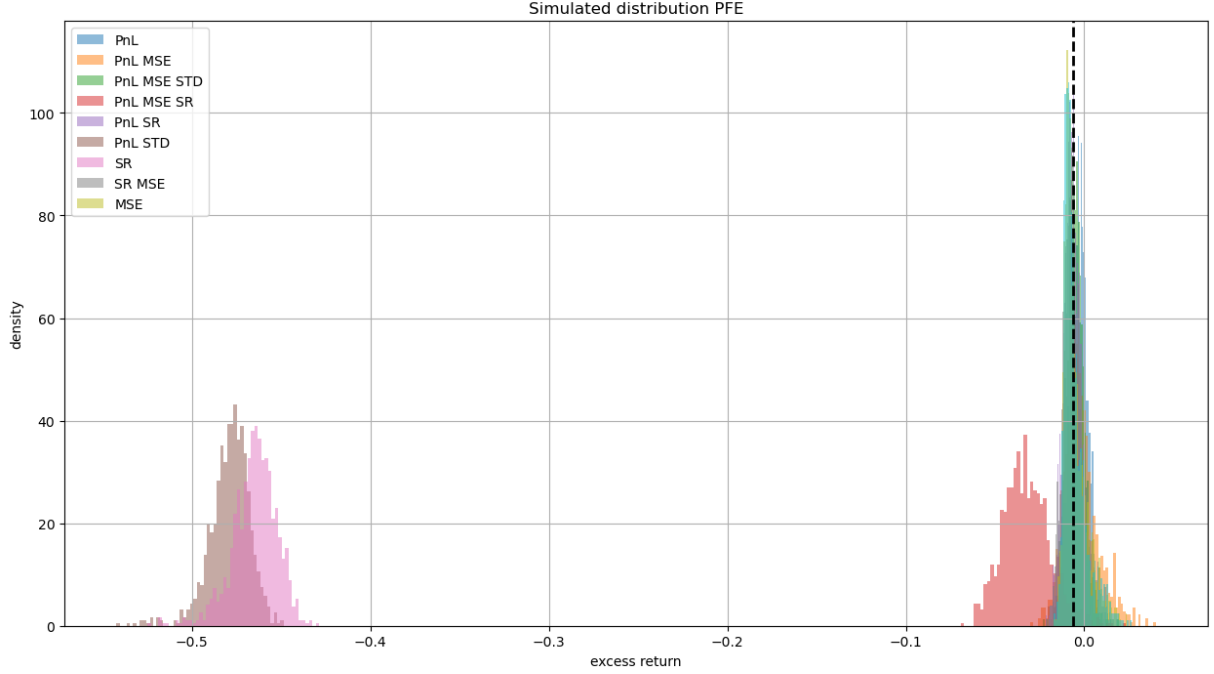


Figure 5.: An illustration of how the Fin-GAN loss function terms can shift generated distributions. All the distributions shown are generated using the same condition window. The black vertical line is the true value, the target. The data used are the ETF-excess returns of PFE.

7. Baseline algorithms

Apart from the standard ForGAN (GAN with the ForGAN architecture trained via the BCE loss), we compare our Fin-GAN model with more standard supervised learning approaches to time series forecasting: ARIMA and LSTM. For completeness, and to demonstrate that the task at hand is non-trivial, we further include PnL and Sharpe Ratio values of long-only strategies, where the predicted sign is +1 for each observation. All comparisons have the same training-validation-testing split as those used in the Fin-GAN experiments.

7.1. ARIMA

We first recall the *auto regressive integrated moving average* (ARIMA), a very classical time series model (Tsay 2005). The parameters p, d, q in $\text{ARIMA}(p, d, q)$ correspond to the autoregressive, differencing, and moving average parts, respectively. The differencing coefficient d indicates how many times we need to difference our initial time series in order to reach a time series X_t which has no unit roots. To determine d , we perform the augmented Dickey-Fuller (ADF) test. In our case, all of the time series had p-values smaller than 10^{-6} , indicating stationarity. This is not surprising, since we are working with returns and excess returns, which are already differenced log-prices time series. Therefore, in our setting, we set $d = 0$ throughout, hence working with $\text{ARMA}(p, q)$ models

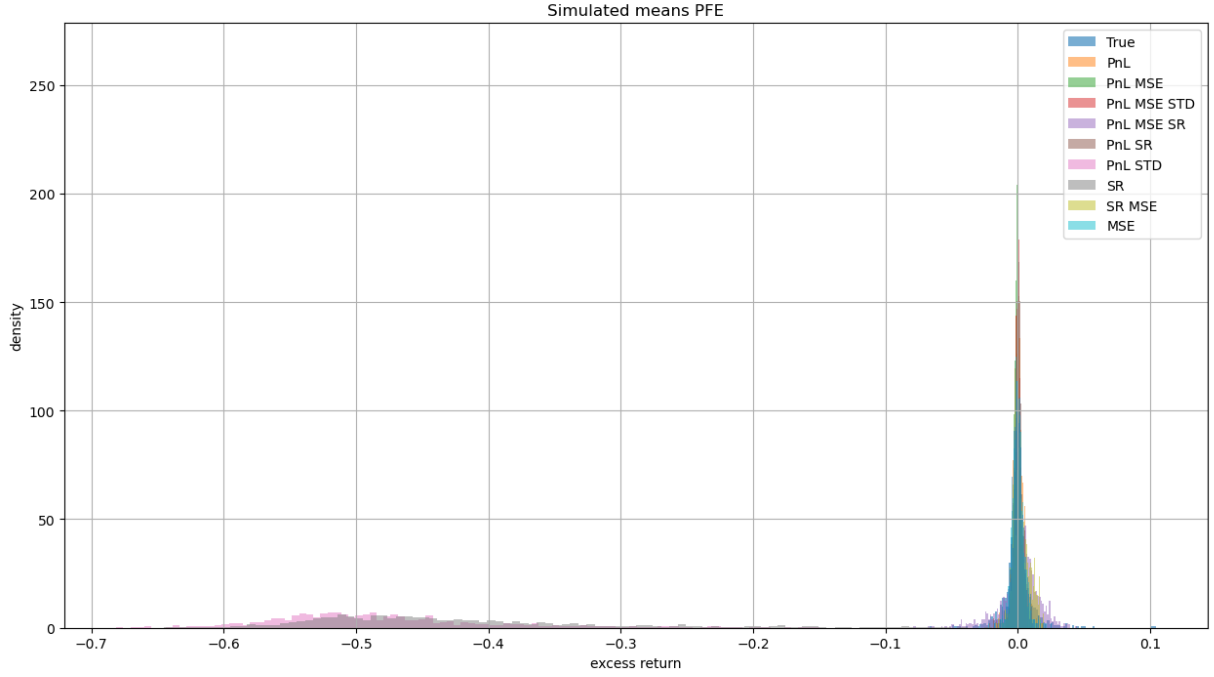


Figure 6.: Out-of-sample (test) simulated means by different Fin-GAN combinations on PFE data. The loss function chosen by the validation is PnL-MSE-SR.

of the form

$$X_t = \alpha_1 X_{t-1} + \cdots + \alpha_p X_{t-p} + \epsilon_t + \theta_1 \epsilon_{t-1} + \cdots + \theta_q \epsilon_{t-q}, \quad (23)$$

where $\epsilon_1, \dots, \epsilon_t$ are white noise terms, θ_i are the moving average parameters, and α_j are the autoregressive parameters. In order to determine the most suitable p and q , we plot the autocorrelation (ACF) and partial autocorrelation (PACF), which indicate that $p, q \in \{0, 1, 2\}$. We fit ARMA(p, q) with $(p, q) \in \{(1, 0), (1, 1), (2, 0), (2, 1), (2, 2)\}$ and choose the model with the lowest Akaike Information Criterion (AIC).

7.2. LSTM

Long short-term memory networks, or LSTMs (Hochreiter and Schmidhuber 1997), belong to the class of recurrent neural networks. They help with the vanishing/exploding gradients problem, and are particularly well-suited for working with time series. At every time t , there is the input $x(t)$, the previous cell state $c(t-1)$ and the previous hidden state $h(t-1)$ (the previous LSTM output). These are then processed by the forget gate, the input gate, and the output gate, in order to create the new cell state $c(t)$ and the new hidden state, the output, $h(t)$. An illustration of the LSTM cell architecture is shown in Figure 7.

For training, we use the same setup as for Fin-GAN. That is, we use RMSProp as the optimiser of choice, train for 125 epochs in total. We are in a regression setting now, so the LSTM is trained to minimise the MSE loss. The rate is once again 0.0001.

LSTM-Fin. We also train LSTM on the appropriate combinations of the (baseline) MSE loss, PnL^* , SR^* , and STD loss terms, in order to have a better comparison with Fin-GAN. We use the same methodology as we did in the GAN setting, also performing gradient norm matching to determine the values of the hyperparameters corresponding to the newly included loss terms.

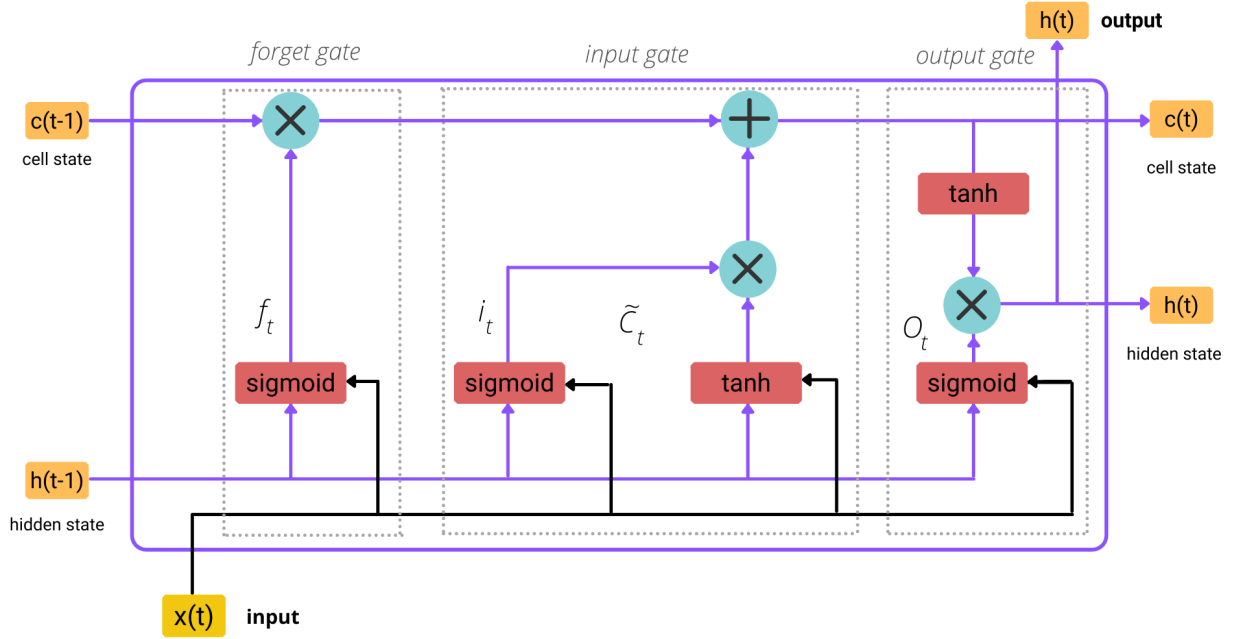


Figure 7.: Illustration of an LSTM cell.

7.3. Long-only strategies

We also remark on the results of long stock and short ETF (for the stock data); and long-only ETF (for the ETF data), which corresponds to constantly forecasting +1 for the sign.

8. Numerical experiments

In this section, we first consider a single-ticker setting. That is, we train Fin-GAN and the baselines on ETF-excess returns/raw returns of a particular stock/ETF. We then explore the notion of universality, in the spirit of [Sirignano and Cont \(2019\)](#), on three different sets of stocks. We pool the data across the tickers used in the single-ticker setting, and train Fin-GAN as if the data was coming from the same source. We repeat this methodology on stocks belonging to the same sector (Consumer Staples), with and without the XLP raw returns used in the training stage. In the universal setting, we further investigate the performance of Fin-GAN on stocks not previously seen by the model during training.

8.1. Single stock & ETF settings

We first examine the summarised performance of the models under consideration: Fin-GAN, For-GAN (trained via the BCE loss), LSTM, LSTM-Fin, ARIMA, and the long-only approach. The statistics of interest are the annualised Sharpe Ratio, mean daily PnL, MAE, and RMSE. All statistics are summarised by computing the mean and the median across the 31 data sets (22 stocks and 9 ETFs) used in the numerical experiments. Additionally, we report on the *portfolio Sharpe Ratio* by first summing the daily Pnls across all tickers in the universe, and then computing the Sharpe Ratio of the resulting daily PnL time series. These summary statistics are shown in Table 2.

We remark that the highest mean, median, and portfolio Sharpe Ratios are achieved by Fin-GAN, followed by LSTM. The median Sharpe Ratio achieved by Fin-GAN is close to twice that of LSTM. Furthermore, we note that despite the Sharpe Ratio being the highest when using the Fin-GAN approach, the highest mean PnLs are achieved by the LSTM, and the highest median PnLs by

	Fin-GAN	ForGAN	LSTM	LSTM-Fin	ARIMA	Long-only
Mean SR	0.540	0.033	0.467	0.341	0.206	0.182
Median SR	0.413	-0.092	0.214	0.170	0.204	0.194
Portfolio SR	2.107	0.172	2.087	0.942	0.612	0.618
Mean PnL	2.978	0.25	4.123	2.361	2.059	2.350
Median PnL	1.890	-0.673	1.959	1.735	2.245	1.975
Mean MAE	0.044	0.052	0.007	0.007	0.007	
Median MAE	0.008	0.009	0.007	0.007	0.007	
Mean RMSE	0.049	0.056	0.012	0.012	0.012	
Median RMSE	0.012	0.014	0.011	0.011	0.011	

Table 2.: Summary of performance metrics over the models across the stocks and ETFs. SR refers to the annualised Sharpe Ratio, and PnL refers to the mean daily PnL. MAE and RMSE represent the mean absolute error and the mean root squared error, respectively. Highlighted are the best-performing results according to each metric.

ARIMA. This altogether demonstrates the significant reduction in variance and higher consistency of the generated PnL by Fin-GAN, compared to the other models. It is also a consequence of placing smaller trades compared to other strategies due to the ability to leverage the uncertainty estimates and implement the weighted strategy. We stress the beneficial effect of the novel loss function terms on performance, evidenced by the mean, median, and portfolio Sharpe Ratios achieved by Fin-GAN being significantly higher than those achieved by ForGAN trained on the BCE loss only.

We further note that ARIMA attains the best performance in terms of RMSE and achieves the best mean MAE, while LSTM achieves the lowest median MAE. We also remark that the non-GAN models have their MAE and RMSE summary statistics on the same scale, while the mean MAE and RMSE are significantly higher for the GAN models. However, the medians are on a similar scale to the other models, and errors are reduced when moving from ForGAN to Fin-GAN. Investigating RMSE and MAE in Figures 13 and 12, respectively, we find that the main cause of such high mean MAE and RMSE achieved by Fin-GAN is performance on IBM. However, Fin-GAN achieves higher Sharpe Ratio (Figure 8) and PnL (Figure 9) than the other models in this case. Similarly, even though ARIMA attains values close to the realised ones, it does so on the opposite side of the real line, when the movements in the underlying are large, resulting in lower PnL and lower Sharpe Ratios. We note that the summary statistics achieved by the non-deep learning approaches, ARIMA and long-only, are similar, and that they achieve better performance than ForGAN trained in a classical way. The long-only column indicates that the task at hand is non-trivial, given that the only ticker on which a Sharpe Ratio above one is achieved is XLK. This is not surprising given the continued increase in stocks belonging to the technology sector during the Covid-19 pandemic. Furthermore, the positive effect that the economics-driven loss function terms had in the GAN setting is not visible in the case of LSTMs.

The above experiments have demonstrated that the Fin-GAN approach outperforms, on average, ForGAN, LSTM, ARIMA, and long-only baselines. We now further investigate the behaviour of Sharpe Ratios at the individual ticker level. The annualised Sharpe Ratio performance across the models and tickers is displayed in Figure 8. We observe that the Fin-GAN model outperforms all other baselines, being able to achieve very competitive Sharpe Ratios, especially in light of the fact that we are dealing with single instrument portfolios. Sharpe Ratios achieved by Fin-GAN are more stable than those of LSTM, and the only ticker with Sharpe Ratio below -1 is *XLY* (Consumer Discretionary). This is not surprising, as this sector ETF had a crash at the start of the Covid-19 pandemic, and was consistently growing during the time used for validation. On the data sets on which LSTM achieves Sharpe Ratios above 1 or 2, Fin-GAN does so as well, achieving

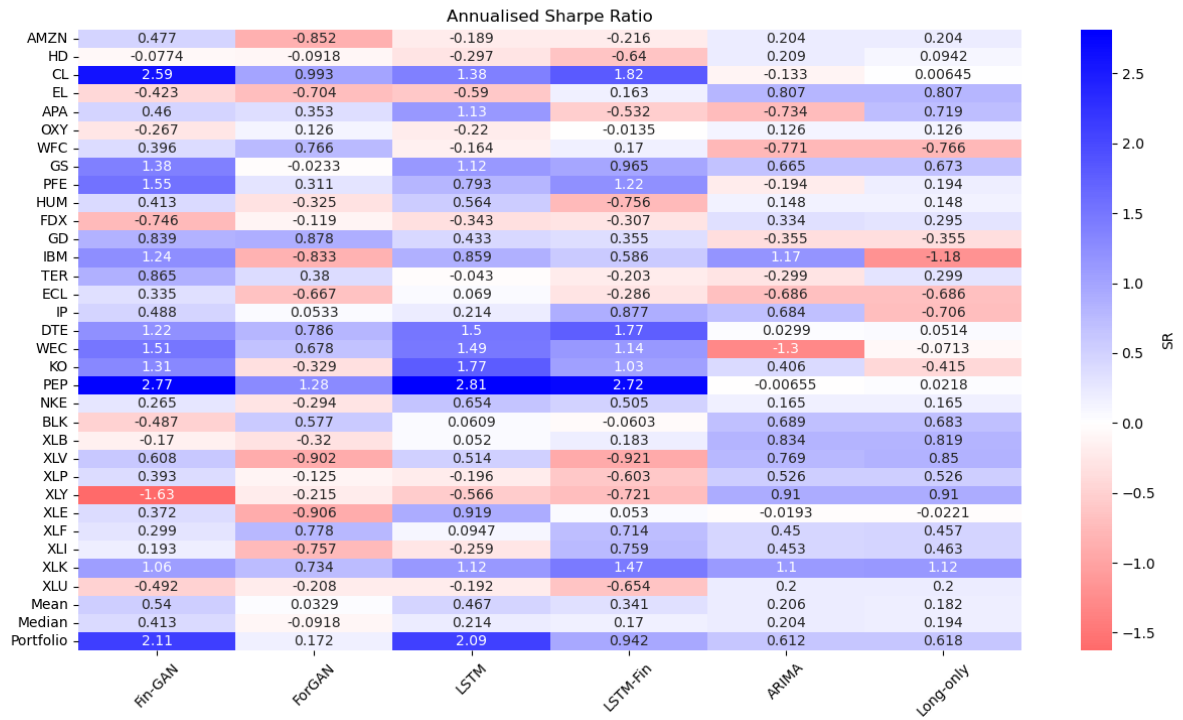


Figure 8.: SR (annualised Sharpe Ratio) obtained by different methods on the test set.

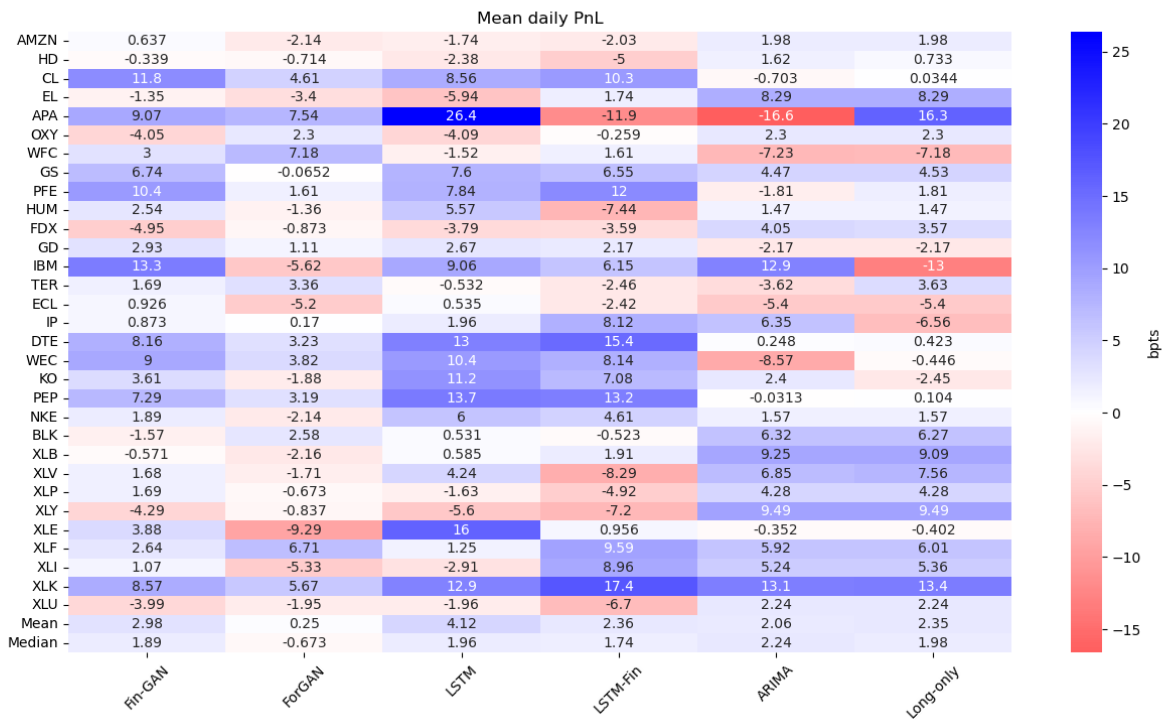


Figure 9.: Mean daily PnL in basis points obtained by different methods.

similar Sharpe Ratios, or a group higher in the case of CL. Altogether, we observe clear benefits from using Fin-GAN when compared to the other methods, in terms of Sharpe Ratio performance. The breakdown of the Fin-GAN cumulative PnL performance by ticker is displayed in Figure 10. We remark that the Covid pandemic had a different effect on different stocks, and that performance on March-June 2020 is the main cause of volatility in the attained Pnls.

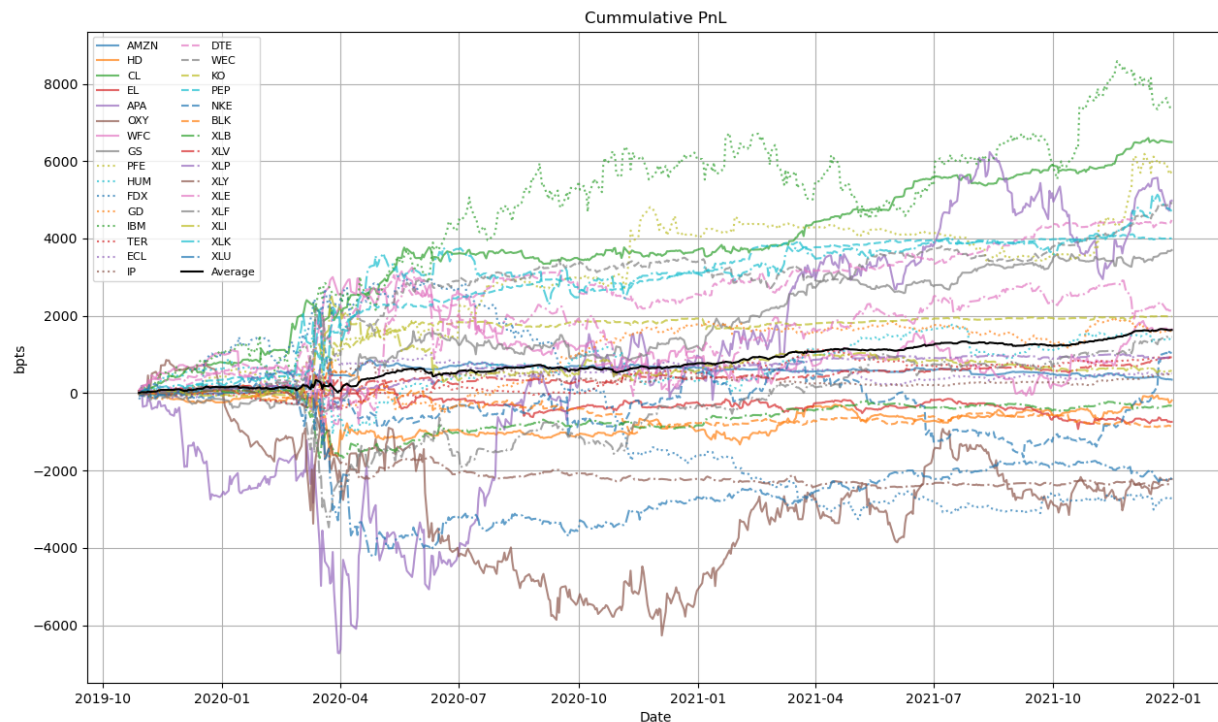


Figure 10.: Cumulative PnL across tickers achieved by different loss function combinations of Fin-GAN. The portfolio PnL is the average PnL displayed in black, multiplied by the number of instruments. A comparison of the overall portfolio performance across the baselines (and the Fin-GAN loss function combinations) is shown in Figure 11.

In terms of the overall performance, Fin-GAN and LSTM outperform all the other methods under consideration. Cumulative PnL plots of the corresponding portfolios are shown in Figure 11. The deep learning models recover from the Covid-19 shock much faster than ARIMA and long-only do, and we note that most of the volatility in the generated Pnls is stemming from the pandemic. In Figure 11 we display the cumulative Pnls achieved by different Fin-GAN loss combinations, including MSE with BCE term only, which on average has a 99.7% correlation with ForGAN. It is evident that the overall performance is increased by using validation, rather than using the same loss combination from the start for every data set. The LSTM has a higher portfolio PnL, but the path is more volatile, especially from March 2020 until June 2020, resulting in a lower Sharpe Ratio than Fin-GAN.

Concerning the mean daily PnL performance displayed in Figure 9, we note that ARIMA and the long-only approach have a tendency of producing mean daily Pnls on the same scale, while LSTM attains the highest Pnls on average. However, the Pnls achieved by LSTM fluctuate significantly from ticker to ticker. The mean daily Pnls achieved by the GAN approaches have a much lower standard deviation (4.85 for Fin-GAN, 4.00 for ForGAN) than LSTM (7.48), LSTM-Fin (7.53), ARIMA (6.26), and long-only (5.97), showcasing the benefits of being able to leverage the

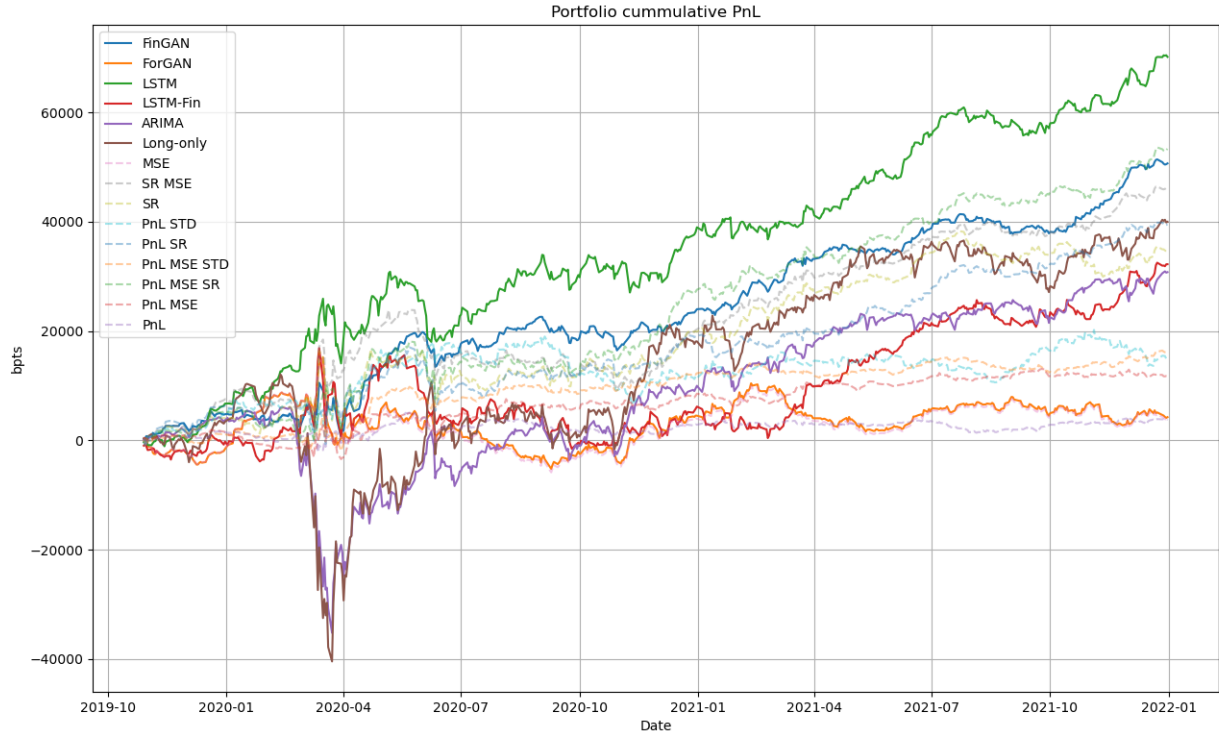


Figure 11.: Portfolio cumulative PnL of different models. Dashed lines correspond to PnL paths generated by the appropriate Fin-GAN loss function combinations (including MSE alone).

uncertainty estimates for developing a weighted strategy with dynamic sizing.

Next, we comment on the breakdown of the chosen loss function combinations in the Fin-GAN performance. As shown in Figure 5, it can be beneficial to utilise the introduced loss function terms together, both for shifting distribution purposes, achieving a better approximation to the data distribution, and for alleviating mode collapse. We remark that the inclusion of the MSE term is very prominent, and that interestingly, the PnL and Sharpe Ratio combination was never chosen. The most common choice was Sharpe Ratio with MSE, in 29% of the cases. The breakdown of the number of times each of the combinations was used is shown in Table 3.

Combination	Count
PnL	3
SR	3
PnL & MSE	5
PnL & SR	0
SR & MSE	9
PnL & MSE & SR	4
PnL & STD	5
PnL & STD & MSE	2

Table 3.: Number of chosen (highest SR on the validation set) Fin-GAN loss function term combinations across the data.

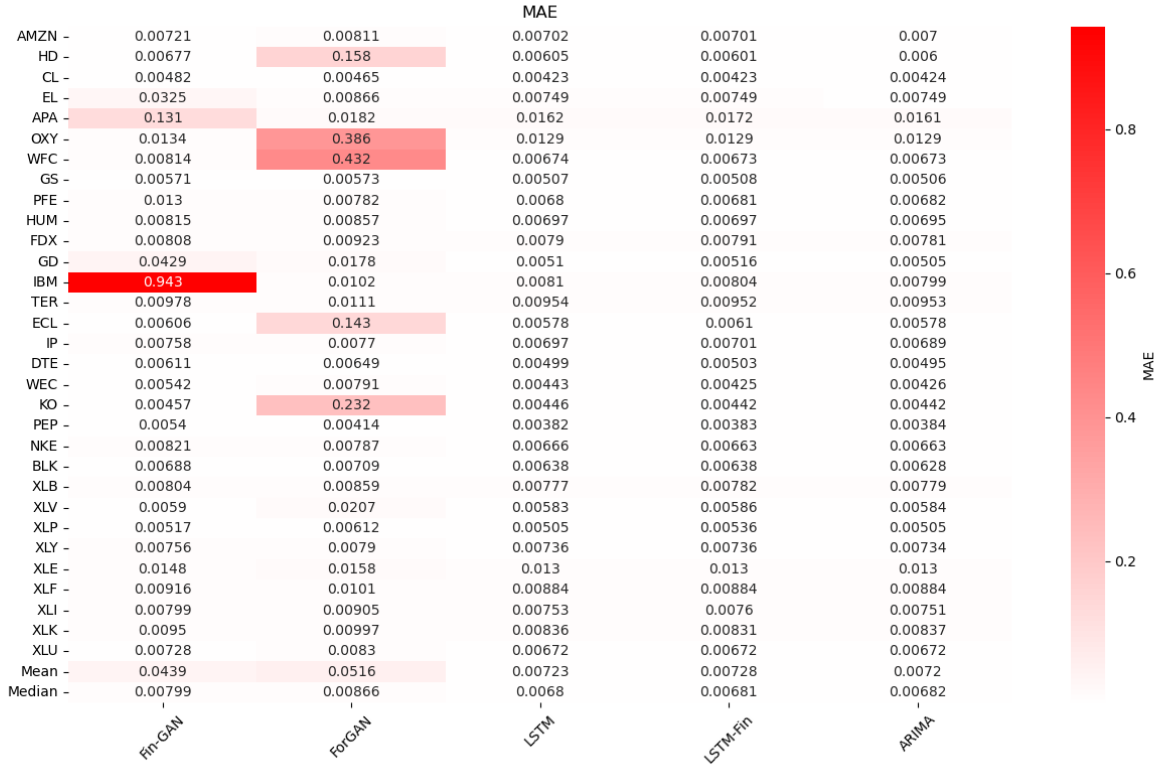


Figure 12.: MAE obtained by different methods.

We compare the Sharpe Ratio performance of the different Fin-GAN loss function combinations on the test set, and show the results in Figure 14. Due to the differences in validation and test set, sometimes a sub-optimal loss function combination is chosen during the validation stage. However, we note that performing validation in order to choose which terms to include in the training objective improves the Sharpe Ratio performance of the model.

As previously discussed, SR and the combination of PnL and STD (PnL & STD) terms convey the same information, but have different gradients, and may result in different forecasts. Hence, we investigate the average Pearson correlation of the obtained out-of-sample PnLs (test set) by different loss term combinations. It is not surprising that there is a high correlation between combinations with the MSE term included, and the corresponding ones with the MSE term excluded. The correlation between the SR term and PnL & STD term is 33.5%, indicating that the two learn very different distributions, while once the MSE term is included, the correlation increases significantly.

An important finding is that the correlation between the MSE term with the BCE loss and the BCE loss is 99.7% on average, implying that ForGAN trained via the binary cross entropy loss is already aiming to produce outputs close to the real values. This behaviour could be simply explained by the structure of the data and the task at hand: since there is a *true target* for each condition, that is, the empirical conditional distribution is point mass $p_{data}(x_{t+1}|x_t^{-L}) = \delta(x_{t+1})$ if L is large enough. Hence, it is not surprising that the forecast values close to the target would receive high scores from the discriminator, encouraging mode collapse in a classical GAN setup. This further supports the decision to use ForGAN as the baseline, its suitability for probabilistic forecasting, and the necessity to customise it to a financial setting.

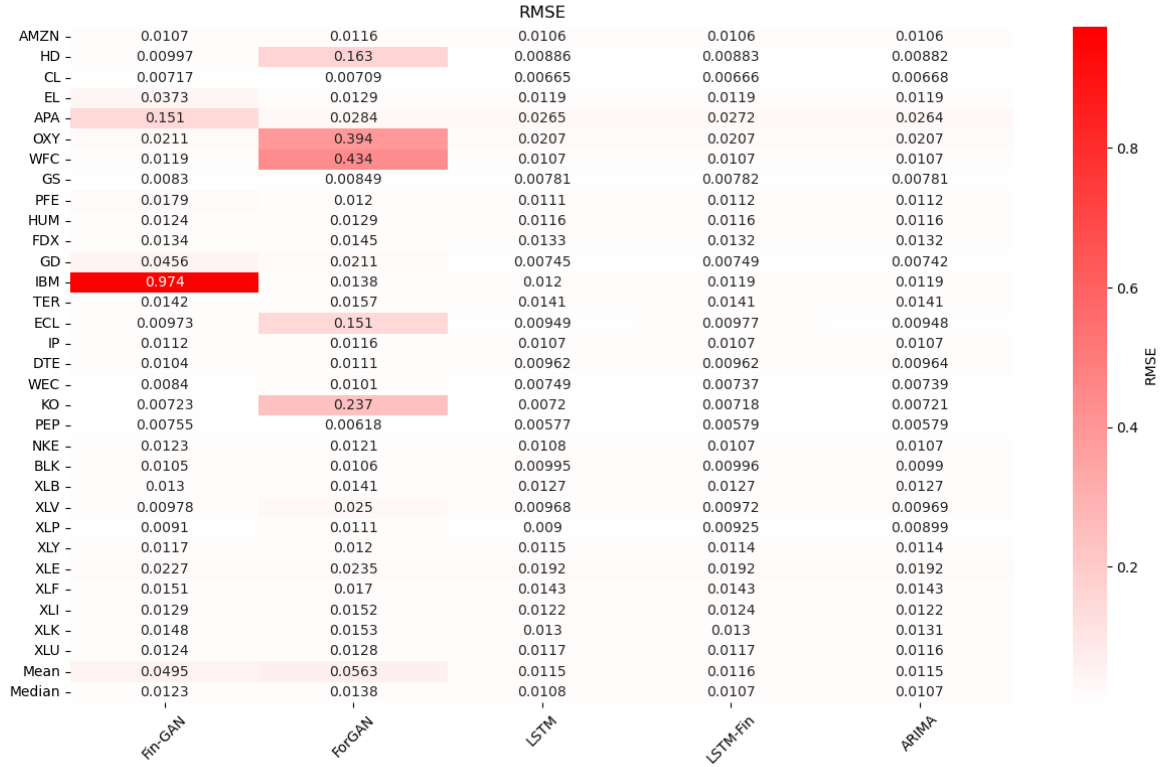


Figure 13.: RMSE obtained by different methods.

8.2. Universality

We test for universality in our approach, in the spirit of [Sirignano and Cont \(2019\)](#). However, due to computational cost, we are only able to perform numerical experiments on a small universe of stocks and ETFs, the 31 previously used. We do not attempt to learn from the cross-asset correlations, but rather consider each time series individually. That is, we employ the single-asset model, where we combine (pool) the data from all 31 tickers. We test the performance of the model on all stocks used for training, and on four additional unseen stocks. The time periods for the training, validation, and test sets across different tickers, as well as the Fin-GAN methodology remain the same. A summary of the Sharpe Ratio performance of the small universal model is shown in the heatmap displayed in Figure 16. The *Single Stock* column refers to the highest Sharpe Ratio achieved by the single-stock model when training on a particular ticker only. Stocks CCL, EBAY, TROW, and CERN have not been seen by the model during the training stage. We note that it is possible to achieve competitive Sharpe Ratios even on unseen stocks. For example, Sharpe Ratios achieved on THROW data are often above 1, despite the fact that the universal model has never seen the EBAY data. Figure 16 indicates that some of the stocks included in the training of the universal model benefit from the pooled training with other stocks, but not all. This is most visible for XLY, where the achieved Sharpe Ratios are either significantly less negative, or even positive. The overall performance not increasing could be due to our universe of stocks being small, and not having strong correlations. One would expect that cross-asset interactions are more informative when stocks belong to the same sector.

We repeat the same analysis using stocks from the Consumer (XLP) sector. We perform one set of numerical experiments with XLP data (raw returns) included in the training data, and another set of experiments without it. The Sharpe Ratio performance without the XLP data included

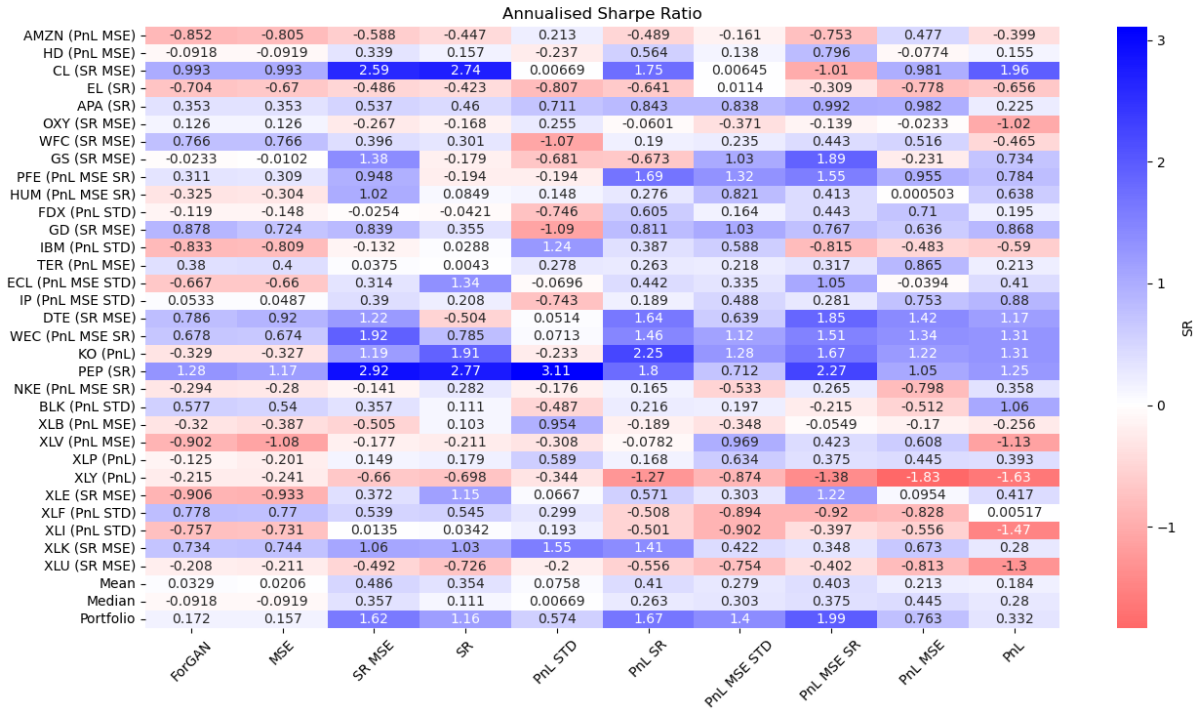


Figure 14.: SR (annualised Sharpe Ratio) obtained by different loss combinations of Fin-GAN on the test set. The chosen loss combination (the combination achieving the highest Sharpe Ratio on the validation set) is reported in parentheses, for each ticker.

is shown in the heatmap displayed in Figure 17, while same performance when the XLP data is included is shown in Figure 18. Stocks unseen by the model are SYY and TSN. Similarly to the previously discussed universal model, Sharpe Ratios achieved on the unseen data can be very competitive, see for example TSN. When XLP data is included, the model chosen by validation is PnL & SR, achieving a good overall portfolio SR of 1.43 on the test set. Although not all stocks have a positive Sharpe Ratio, this combination achieves good and even very good SRs, including the Sharpe Ratio of 1.55 on unseen TSN data. An important observation is that the Sharpe Ratio on XLP data, in this case, is 1.11, compared to 0.393 in the single-stock setting. Other ETFs might as well benefit from being trained alongside their constituents. When XLP data is removed from the pool of stocks used for training, the model of choice is SR. The portfolio Sharpe Ratio reduces to 1.18, but remains competitive. Comparing the heatmaps in Figures 17 and 18, we conclude that there is a clear benefit from including the ETF data in the training process of Fin-GAN, but not of ForGAN. Training alongside the XLP data results in more stable Sharpe Ratios compared to the model without it.

9. Conclusion and future outlook

We have shown that GANs can be successfully employed for probabilistic time series forecasting in the context of financial data, where the directionality (sign) of the forecast is of main interest, particularly ahead of large price jumps. We introduced a novel economics-driven generator loss function, which includes suitably weighted Profit and Loss (PnL), standard deviation of the PnL, MSE, Sharpe Ratio-based loss function terms, rendering GANs more suitable for a classification task, and placing them into a supervised learning setting.

In a comprehensive set of numerical experiments, we compared our approach to a standard GAN

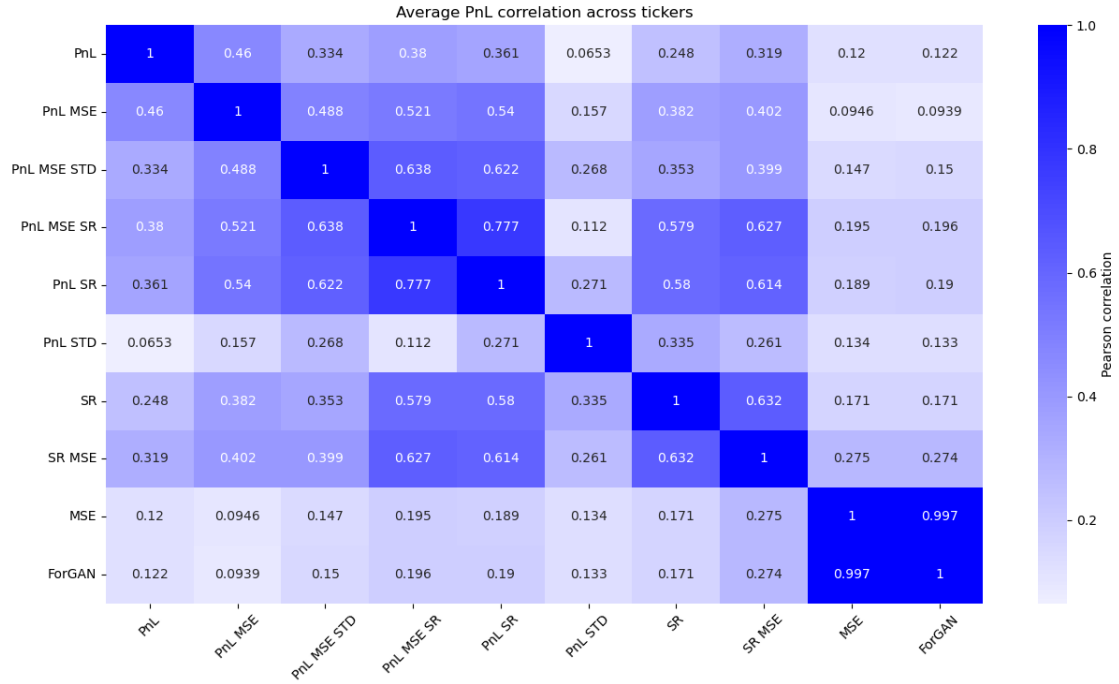


Figure 15.: Mean out-of-sample (test) correlation of PnLs across different tickers of the Fin-GAN loss term combinations.

model, LSTM, ARIMA and long-only strategies, and demonstrated superior performance of our model in terms of Sharpe Ratios achieved.

Furthermore, we considered a universal model in three settings: pooling the data from stocks belonging to different sectors and sector ETFs, considering one sector with the sector ETF included in the training data and with the sector ETF excluded from the training stage. Even though the universe of stocks was small, we note that good performance is possible even on unseen stocks. Furthermore, performance in the single-sector setting increased when the sector ETF was included in the training data.

Our proposed Fin-GAN methodology was shown to be able to significantly improve Sharpe Ratio performance, shift generated distributions, as well as help alleviate mode collapse issues, the latter of which is a standard challenge in many GAN-based approaches.

Our work raises a number of interesting future directions. It could be insightful to explore how the addition of the new loss function terms impacts path simulations, and how to leverage cross-asset interactions, learn from correlations and explore the notion of universality in a larger universe of stocks. Moving beyond equity data and exploring other classes of assets (such as options), potentially in a joint framework, is another interesting avenue for future work. Our proposed model and architecture could benefit from being trained on higher-frequency data, for eg, intraday minutely data. An additional area of interest would be to explore how one might combine the Fin-GAN loss with COT-GAN (Xu et al. 2020) and Time-GAN (Yoon et al. 2019) Exploring more complex architectures, a larger hyperparameter space, and longer training could further yield improved results.

Acknowledgements

We thank BNP Paribas for their generous support of Milena Vuletić’s research. We also thank the anonymous referees, Justin Sirignano, and audiences from the Mathematical Institute and Depart-

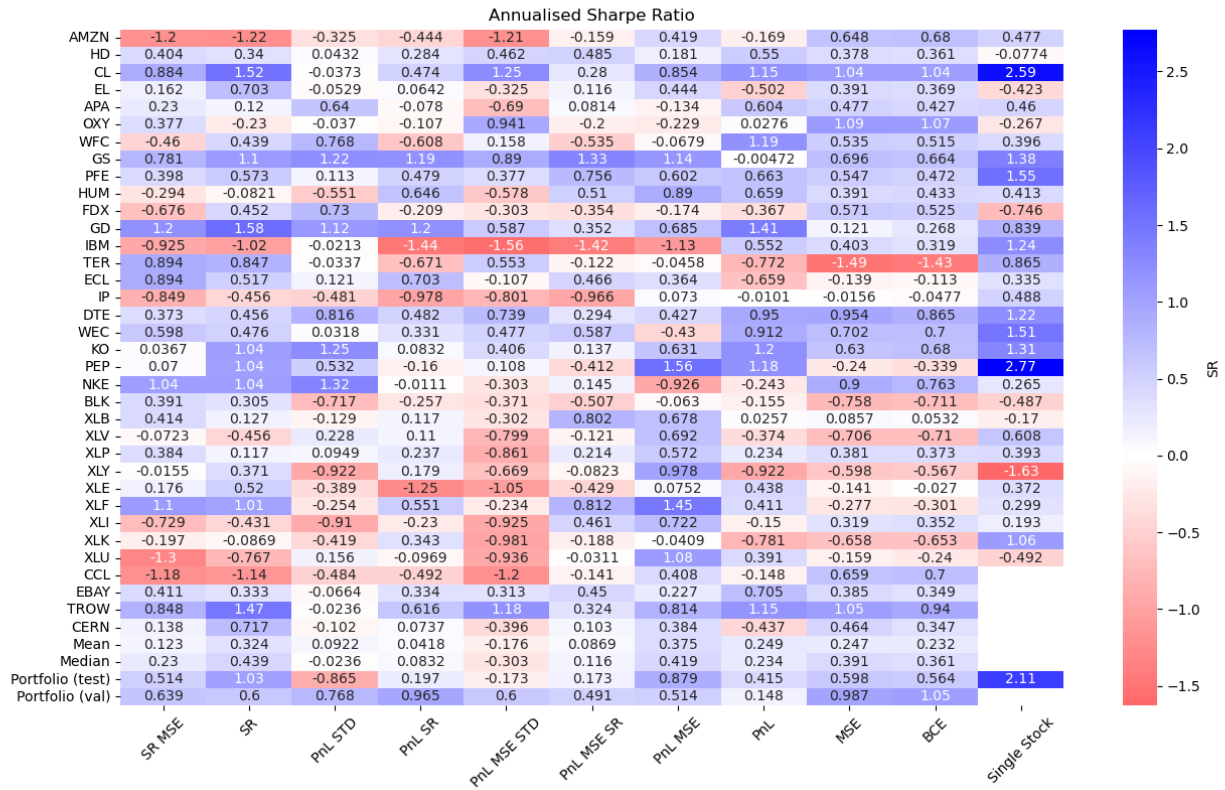


Figure 16.: Summary of Sharpe Ratio performance for individual stocks in the universal model. Each column represents a different combination of the loss function terms. The *Single Stock* column shows the best Fin-GAN performance when trained on a particular stock/etf. CCL, EBAY, TROW and CERN have not been seen by the model during the training stage.

ment of Statistics at the University of Oxford, Oxford-Man Institute of Quantitative Finance, ETH Zurich, and Man AHL, whose feedback and comments have improved our work.

Funding

This publication is based on work supported by the EPSRC Centre for Doctoral Training in Mathematics of Random Systems: Analysis, Modelling and Simulation (EP/S023925/1).

Data availability

The data used is extracted from CRSP on Wharton Research Data Services and is available [here](#).

Code availability

The code is implemented in PyTorch and is available on request from the corresponding author.

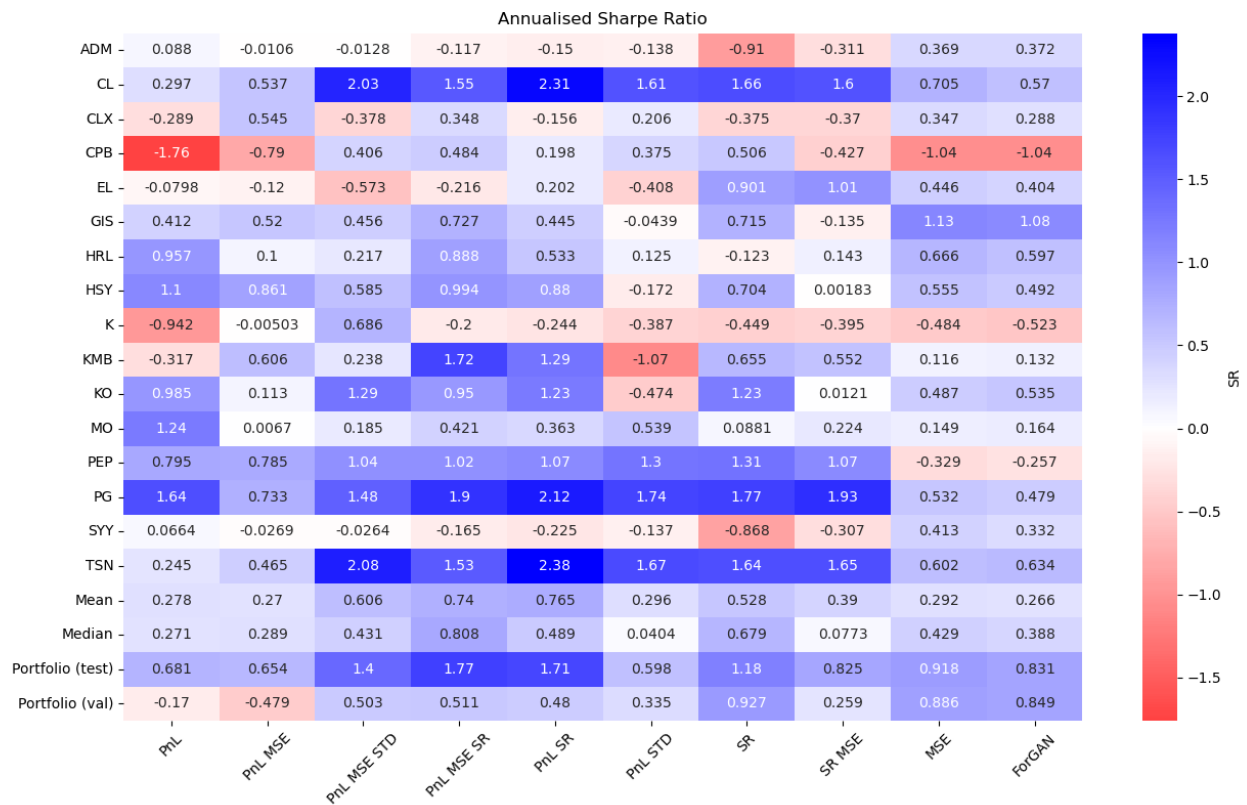


Figure 17.: Summary of SR performance on stocks constituents of the XLP sector. Each column represents a different combination of loss function term. SY and TSN have not been seen by the model during the training stage.

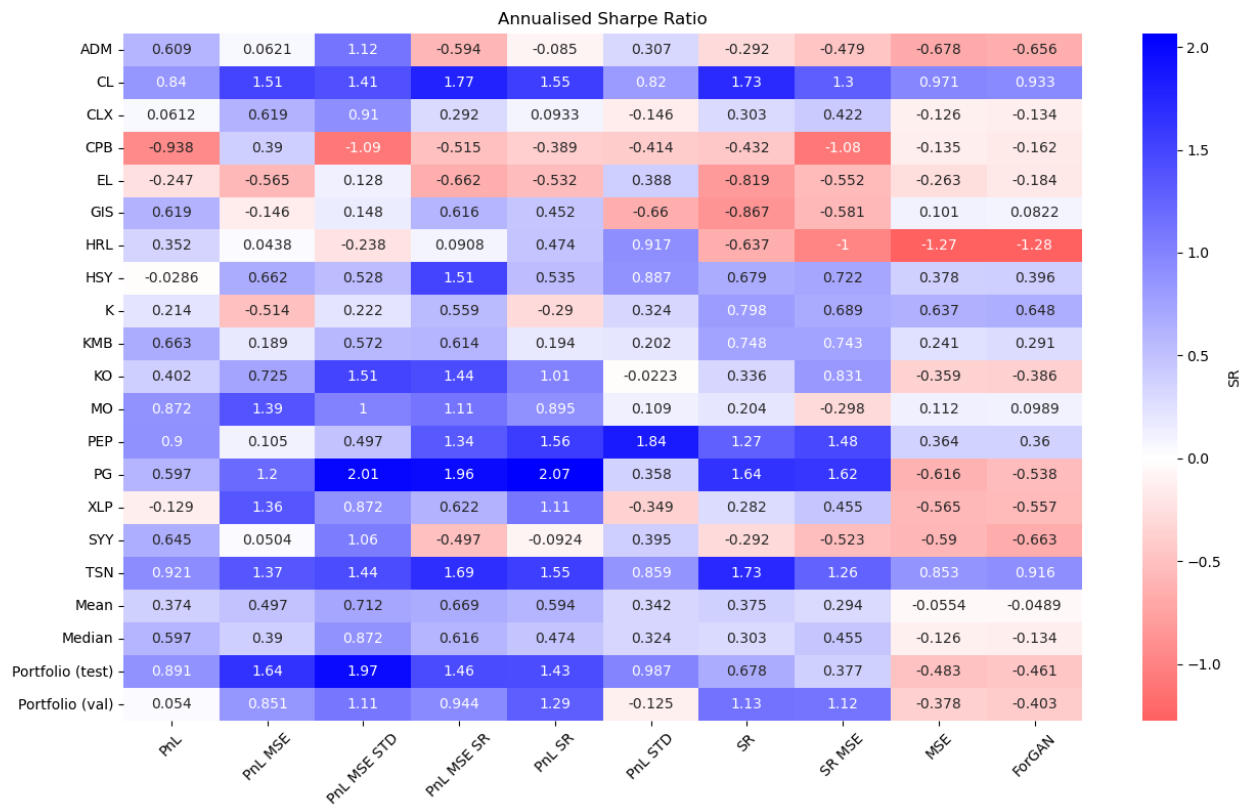


Figure 18.: Summary of SR performance on the stocks belonging to the XLP sector, with XLP data included in the training data. Each column represents a different combination of loss function terms. SY and TSN have not been seen by the model during the training stage.

References

- M. Arjovsky and L. Bottou. Towards Principled Methods for Training Generative Adversarial Networks. In *International Conference on Learning Representations*, 2017.
- M. Arjovsky, S. Chintala, and L. Bottou. Wasserstein Generative Adversarial Networks. In *International Conference on Machine Learning*, pages 214–223. PMLR, 2017.
- S. A. Assefa, D. Dervovic, M. Mahfouz, R. E. Tillman, P. Reddy, and M. Veloso. Generating synthetic data in finance: opportunities, challenges and pitfalls. In *Proceedings of the First ACM International Conference on AI in Finance*, pages 1–8, 2020.
- S. Bhatia, A. Jain, and B. Hooi. ExGAN: Adversarial Generation of Extreme Samples. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(8):6750–6758, May 2021. .
- G. E. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung. *Time series analysis: forecasting and control*. John Wiley & Sons, 2015.
- H. Buehler, L. Gonon, J. Teichmann, and B. Wood. Deep hedging. *Quantitative Finance*, 19(8):1271–1291, 2019.
- H. Buehler, B. Horvath, T. Lyons, I. Perez Arribas, and B. Wood. Generating financial markets with signatures. *Available at SSRN 3657366*, 2020.
- T. Chen and C. Guestrin. Xgboost: A scalable tree boosting system. In *KDD*, 2016.
- K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, Doha, Qatar, Oct. 2014. Association for Computational Linguistics.
- R. Cont and M. Vuletić. Simulation of arbitrage-free implied volatility surfaces. *Available at SSRN 4299363*, 2022.
- R. Cont, M. Cucuringu, R. Xu, and C. Zhang. Tail-GAN: Nonparametric Scenario Generation for Tail Risk Estimation. Papers 2203.01664, arXiv.org, Mar. 2022.
- A. V. Dorogush, V. Ershov, and A. Gulin. CatBoost: gradient boosting with categorical features support. *arXiv preprint arXiv:1810.11363*, 2018.
- R. Durall, A. Chatzimichailidis, P. Labus, and J. Keuper. Combating Mode Collapse in GAN training: An Empirical Analysis using Hessian Eigenvalues. In *VISIGRAPP*, pages 211–218, 01 2021. .
- L. Galteri, L. Seidenari, M. Bertini, T. Uricchio, and A. Del Bimbo. Fast Video Quality Enhancement using GANs. In *Proceedings of the 27th ACM International Conference*, pages 1065–1067, 10 2019. ISBN 978-1-4503-6889-6. .
- X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.
- I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative Adversarial Nets. In *Advances in Neural Information Processing Systems*, volume 27, 2014.
- I. Goodfellow, Y. Bengio, and A. Courville. *Deep learning*. MIT press, 2016.
- I. J. Goodfellow. NIPS 2016 Tutorial: Generative Adversarial Networks. *ArXiv*, abs/1701.00160, 2017.
- I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. C. Courville. Improved Training of Wasserstein GANs. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- E. Guresen, G. Kayakutlu, and T. U. Daim. Using artificial neural network models in stock market index prediction. *Expert Systems with Applications*, 38(8):10389–10397, 2011.
- K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015.
- G. Hinton, N. Srivastava, and K. Swersky. Neural networks for machine learning, lecture 6. Coursera, 2012.
- S. Hochreiter and J. Schmidhuber. Long Short-term Memory. *Neural Computation*, 9:1735–80, 12 1997. .
- F. Huszár. How (not) to train your generative model: Scheduled sampling, likelihood, adversary? *ArXiv*, abs/1511.05101, 2015.
- G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *NeurIPS*, 2017.
- A. Koochali, P. Schichtel, A. Dengel, and S. Ahmed. Probabilistic Forecasting of Sensory Data With Generative Adversarial Networks–ForGAN. *IEEE Access*, 7:63868–63880, 2019.

- A. Koshiyama, N. Firoozye, and P. Treleaven. Generative adversarial networks for financial trading Strategies fine-tuning and combination. *Quantitative Finance*, 21:1–17, 09 2020. .
- T. Leangarun, P. Tangamchit, and S. Thajchayapong. Stock Price Manipulation Detection using Generative Adversarial Networks. In *2018 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 2104–2111, 2018. .
- J. Li, X. Wang, Y. Lin, A. Sinha, and M. Wellman. Generating realistic stock market order streams. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 727–734, 2020.
- L. Lucchese, M. Pakkanen, and A. Veraart. The short-term predictability of returns in order book markets: a deep learning perspective, 2022.
- G. Mariani, Y. Zhu, J. Li, F. Scheidegger, R. Istrate, C. Bekas, and A. C. I. Malossi. PAGAN: Portfolio Analysis with Generative Adversarial Networks. Papers 1909.10578, arXiv.org, Sept. 2019.
- L. R. Medsker and L. Jain. Recurrent neural networks. *Design and Applications*, 5:64–67, 2001.
- M. Mirza and S. Osindero. Conditional Generative Adversarial Nets. *ArXiv*, abs/1411.1784, 2014.
- M. Mustafa, D. Bard, W. Bhimji, Z. Lukić, R. Al-Rfou, and J. M. Kratochvil. CosmoGAN: creating high-fidelity weak lensing convergence maps using generative adversarial networks. *Computational Astrophysics and Cosmology*, 6(1), May 2019. ISSN 2197-7909. .
- S. Nowozin, B. Cseke, and R. Tomioka. f-GAN: Training Generative Neural Samplers using Variational Divergence Minimization. *Advances in Neural Information Processing Systems*, 29, 2016.
- F. Prenzel, R. Cont, M. Cucuringu, and J. Kochems. Dynamic Calibration of Order Flow Models with Generative Adversarial Networks. In *3rd ACM International Conference on AI in Finance*, pages 446–453, 2022.
- A. N. Refenes, A. Zapranis, and G. Francis. Stock performance modeling using neural networks: a comparative study with regression models. *Neural networks*, 7(2):375–388, 1994.
- S. Roberts, M. Osborne, M. Ebden, S. Reece, N. Gibson, and S. Aigrain. Gaussian processes for time-series modelling. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 371(1984):20110550, 2013. .
- R. A. C. Romero. Generative Adversarial Network for Stock Market Price Prediction, 2018. CS230: Deep Learning, Winter 2018, Stanford University, CA.
- T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, X. Chen, and X. Chen. Improved Techniques for Training GANs. In *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016.
- J. Sirignano and R. Cont. Universal features of price formation in financial markets: perspectives from deep learning. *Quantitative Finance*, 19(9):1449–1459, 2019. ISSN 1469-7688. .
- J. A. Sirignano. Deep learning for limit order books. *Quantitative Finance*, 19(4):549–570, 2019.
- S. Takahashi, Y. Chen, and K. Tanaka-Ishii. Modeling financial time-series with generative adversarial networks. *Physica A: Statistical Mechanics and its Applications*, 527:121261, 2019.
- A. Tsantekidis, N. Passalis, A. Tefas, J. Kannianen, M. Gabbouj, and A. Iosifidis. Forecasting stock prices from the limit order book using convolutional neural networks. In *2017 IEEE 19th conference on business informatics (CBI)*, volume 1, pages 7–12. IEEE, 2017a.
- A. Tsantekidis, N. Passalis, A. Tefas, J. Kannianen, M. Gabbouj, and A. Iosifidis. Using deep learning to detect price change indications in financial markets. In *2017 25th European Signal Processing Conference (EUSIPCO)*, pages 2511–2515. IEEE, 2017b.
- R. S. Tsay. *Analysis of Financial Time Series*. John Wiley & Sons, 2005.
- J. Wallbridge. Transformers for limit order books. *arXiv preprint arXiv:2003.00130*, 2020.
- X. Wang, K. Yu, S. Wu, J. Gu, Y. Liu, C. Dong, Y. Qiao, and C. C. Loy. ESRGAN: Enhanced Super-Resolution Generative Adversarial Networks. In L. Leal-Taixé and S. Roth, editors, *Computer Vision – ECCV 2018 Workshops*, pages 63–79, Cham, 2019. Springer International Publishing.
- M. Wiese, R. Knobloch, R. Korn, and P. Kretschmer. Quant GANs: deep generation of financial time series. *Quantitative Finance*, 20(9):1419–1440, Apr 2020. ISSN 1469-7696. .
- Q. Wu, C. G. Brinton, Z. Zhang, A. Pizzoferrato, Z. Liu, and M. Cucuringu. Equity2Vec: End-to-End Deep Learning Framework for Cross-Sectional Asset Pricing. In *Proceedings of the Second ACM International Conference on AI in Finance*, ICAIF ’21, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450391481. .
- Q. Wu, J. Li, Z. Liu, Y. Li, and M. Cucuringu. Symphony in the Latent Space: Provably Integrating High-dimensional Techniques with Non-linear Machine Learning Models. In *Proceedings of the 2023 AAAI Conference on Artificial Intelligence*, 2023.

- Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 2020.
- T. Xu, L. K. Wenliang, M. Munn, and B. Acciaio. COT-GAN: Generating sequential data via causal optimal transport. *Advances in Neural Information Processing Systems*, 33:8798–8809, 2020.
- J. Yoon, D. Jarrett, and M. Van der Schaar. Time-series generative adversarial networks. *Advances in Neural Information Processing Systems*, 32, 2019.
- H. Zhang, V. Sindagi, and V. M. Patel. Image De-Raining Using a Conditional Generative Adversarial Network. *IEEE Transactions on Circuits and Systems for Video Technology*, 30(11):3943–3956, 2020. .
- K. Zhang, G. Zhong, J. Dong, S. Wang, and Y. Wang. Stock Market Prediction Based on Generative Adversarial Networks. *Procedia Computer Science*, 147:400–406, 2019a. ISSN 1877-0509. . 2018 International Conference on Identification, Information and Knowledge in the Internet of Things.
- Z. Zhang and S. Zohren. Multi-horizon forecasting for limit order books: Novel deep learning approaches and hardware acceleration using intelligent processing units. *arXiv preprint arXiv:2105.10430*, 2021.
- Z. Zhang, S. Zohren, and S. Roberts. Deeplob: Deep convolutional neural networks for limit order books. *IEEE Transactions on Signal Processing*, 67(11):3001–3012, 2019b.
- X. Zhou, Z. Pan, G. Hu, S. Tang, and C. Zhao. Stock Market Prediction on High-Frequency Data Using Generative Adversarial Nets. *Mathematical Problems in Engineering*, 2018:1–11, 04 2018. .